

PROTOTYPING A LOW-COST EEG MACHINE

Xavier Asghar - MEng Final Project Report - 2025/26

Abstract Word Count: 248

Body Word Count: 12,402

Supervisor: Dr Nic Zhang

Assessor: Professor John Taylor

Student Number: 219268967

Integrated Mechanical and Electrical Engineering

Ethical Approval has been Received for this Project



UNIVERSITY OF
BATH

EXECUTIVE SUMMARY

Brain-computer interfaces (BCIs) have an expanding range of applications, from assistive technologies for individuals with motor impairments, to autonomous control of mechatronic systems. The increasing availability of commercial EEG devices, from companies such as OpenBCI and Emotiv, has helped make BCI research more accessible outside of clinical and laboratory settings. However these commercial offerings remain prohibitively expensive, often retailing for approximately £1000, and some requiring further subscriptions to access raw data. This high cost has largely excluded EEG technology from educational environments, where it could serve as a platform for student research projects, or as a teaching tool, not just for BCIs, but for signal processing, feature extraction and machine learning.

This report presents the design and validation of a low-cost, open-source EEG module, intended to be easily reproducible. The system utilises the Texas Instruments ADS1299 EEG analogue front end to record high-fidelity EEG signals, alongside an ESP32-S3 microcontroller to wirelessly transmit data to a host computer using Bluetooth Low Energy. A python bridge then pushes the data to the Lab Streaming Layer, enabling real-time visualisation, time-synchronised recording, and integration with established EEG tools. The system records up to 8-channels of 24-bit EEG data at 500 samples per second with no packet loss, and has been validated through the successful detection of alpha waves, SSVEPs, and P300 responses; features that often form the base of BCI systems. With a 4-channel component cost of just £68, the module brings high quality EEG to a cost feasible for educational deployment.

[248 out of a maximum of 250 words]

TABLE OF CONTENTS

01	List of Symbols	3
02	Introduction Project Context • Project Aims • Scientific Background	4
03	Literature Review Commercial Offerings • Academic Prototypes	8
04	System Design Architecture • Firmware • Software • PCB • Mechanical	10
05	System Verification EMG • ECG • Alpha Waves • SSVEPs • P300s	25
06	Discussions & Conclusions Objectives • Application • Limitations • Improvements • Conclusion	33
07	Project Review	38
08	Acknowledgements	40
09	References	41
10	Appendices	46

1. LIST OF SYMBOLS

1.1	List of Acronyms
ADC	Analog to Digital Converter
BCI	Brain Computer Interface
BLE	Bluetooth Low Energy
BOM	Bill of Materials
CMRR	Common Mode Rejection Ratio
DRL	Driven Right Leg
ECG	Electrocardiogram
EEG	Electroencephalography
EMG	Electromyography
EMI	Electromagnetic Interference
ENIG	Electroless Nickel Immersion Gold
EP	Evoked Potential
ERP	Event Related Potential
ESD	Electrostatic Discharge
FFT	Fast Fourier Transform
GUI	Graphical User Interface
HASL	Hot Air Solder Levelling
IC	Integrated Circuit
ISR	Interrupt Service Routine
LDO	Low-Dropout Regulator
LSL	Lab Streaming Layer
MUAP	Motor Unit Action Potential
PCB	Printed Circuit Board
RTI	Referred to Input
SNR	Signal to Noise Ratio
SPI	Serial Peripheral Interface
SPS	Samples Per Second
SSVEP	Steady State Visually Evoked Potential
TI	Texas Instruments
VEP	Visually Evoked Potential
XDF	eXtensible Data Format

2. INTRODUCTION

2.1. Project Context

Electroencephalography (EEG) is a non-invasive method of recording spontaneous electrical activity, originating from summed synchronised synaptic activities in populations of cortical neurons in the brain [1, p. 7]. EEG measurements are taken by placing electrodes at multiple locations on the patient's scalp, and recording the subsequent voltage fluctuations measured at each location. While traditionally used primarily in clinical neurophysiology to monitor and diagnose conditions such as epilepsy, sleep disorders, and brain injuries [2], technological advancements in recent decades have opened up a wide range of new applications involving brain-computer interfaces (BCIs). BCIs use real-time EEG data to control mechanical and electronic devices, with applications including assisting individuals with disabilities and motor impairments, autonomous navigation of mechatronic systems, as well as neurogaming and entertainment [3].

In recent years EEG technology has become more widely available outside of a lab setting, with companies like Emotiv, OpenBCI, Muse, and NeuroSky producing consumer friendly EEG packages that have reduced the barriers to conducting EEG related research [4]. Some of these headsets are shown in Figure 1. However, these devices, which are typically intended to be sold directly to consumers, produce results of varying accuracy, with increased levels of noise [5]. Additionally, a premium is often charged for these devices, as well as for access to proprietary software to view raw EEG data [6], with the more advanced headsets, like OpenBCI's Ultracortex, or Emotiv's EPOC X both costing upwards of \$1000 [7] [8]. This is in spite of the fact that the core components used to measure biopotentials are not overly expensive, nor are the engineering principles behind these devices overly complex [9].

The high cost of consumer EEG headsets has largely prevented the deployment of EEG technology in educational settings [10] to teach students the fundamentals of brain-computer interfaces. However, by utilising standalone electrodes, rather than relying on fully integrated headset systems, the costs can be brought down to a level feasible for educational deployment. Therefore, this project seeks to develop a low-cost, easily reproducible, and open-source EEG module designed specifically to be used as a teaching tool, as well as for use in future EEG related final year projects.

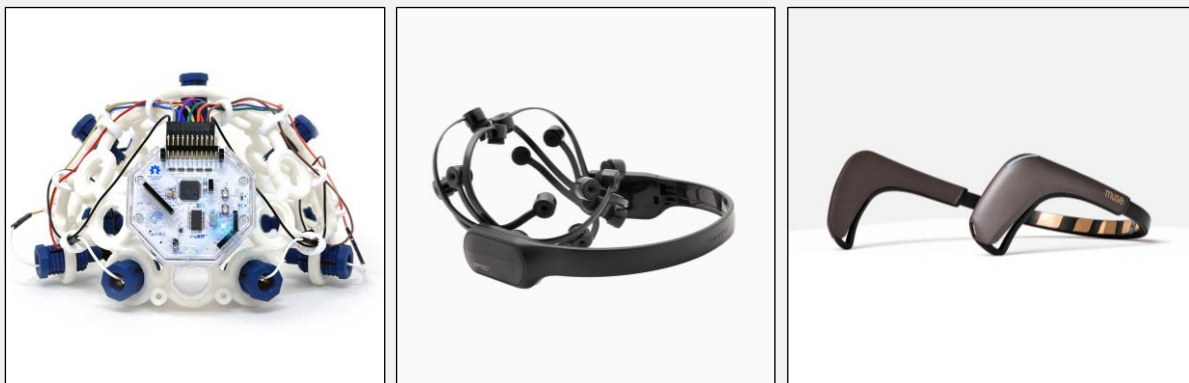


Figure 1: Commercial EEG offerings, from left to right: OpenBCI Ultracortex [7], Emotiv EPOC X [8], Muse 2 [11]

2.2. Project Aims

The primary aim of this project is to use low cost, off the shelf components to design, build, and validate a prototype EEG module. The module must be capable of safely measuring known EEG features, and performing basic data acquisition and signal processing, ensuring the data is easily accessible to the end user. Ultimately the system must be fully documented so it can serve as a reproducible, open-source, educational tool for future brain computer interface research.

It is important to note that several examples of broadly similar low-cost EEG solutions can be found in the literature, as detailed in section 3. However, whilst these papers serve as strong proofs of concept, they fail to provide the reader with sufficient resources to recreate their work, lacking fully realised electrical schematics and code files. The goal of this project is to not just create a tool that can be used within the university, but also can be easily recreated by anyone looking to conduct EEG related research themselves.

To achieve these aims, the project can be broadly divided into four primary objectives, described in Table 1.

Table 1: Project Objectives

Objective 1: Hardware Design and Assembly
Design a low cost printed circuit board (PCB) capable of accurately acquiring EEG measurements from input electrodes, and wirelessly sending the EEG data to an external computer. The PCB must be battery powered, and fully isolated from mains to ensure patient safety.
Objective 2: Software and Firmware Integration
Develop a robust data transmission pipeline, capable of reading data from the analogue front end, wirelessly transmitting said data to a computer, and providing the user with real-time viewing and time-synchronised recording.
Objective 3: System Validation
Test the hardware and software stack to verify its accuracy and safety. This involves conducting human trials to identify known EEG features, including alpha waves, SSVEPs, and P300s
Objective 4: Open-Source Documentation
Produce a comprehensive set of documentation, including a user guide, PCB schematics and Gerber files, commented firmware code and python analysis scripts in order to facilitate future scaling up of the prototype, as well as use and modification in other projects.

2.3. Scientific Background

EEG signals are miniscule in magnitude, typically in the order of microvolts [1, p. 26], due to the layers of cerebrospinal fluid, meninges, bone, and skin they must traverse prior to reaching the electrodes on the patient's scalp [12]. Therefore, recording EEG data necessitates equipment with high levels of precision, as well as adherence to proper procedure when applying the electrodes to a patient. The recorded data originates primarily from the cerebral cortex, the outermost layer of the brain, which contains 14 to 16 billion neurons [13]. This makes the isolation of signals originating from specific neurons impossible, and thus the voltage fluctuation measured at the scalp represents the sum of activities generated from many neural sources surrounding the electrodes [1, p. 8].

Typically, a minimum of three electrodes are required to record EEG data: an active electrode, a reference electrode, and a bias electrode, where the measured voltage is the potential difference between the active, and reference electrode [1, p. 11]. The bias electrode biases the patient such that the common-mode voltage of the active and reference electrode stays within the dynamic range of the EEG circuitry, preventing saturation of the output. More advanced EEG systems also use the bias electrode as part of a noise cancelling feedback loop, called a driven right leg (DRL), where common-mode noise measured from the electrodes, is inverted and fed back into the patient. This helps eliminate the mains-frequency interference that is often present in EEG measurements. [14, p. 68].

The electrodes are most commonly placed on the scalp at some or all of the locations set out by the 10-20 system, shown in Figure 2. This system splits the skull into increments of 10% and 20%, and identifies each location with a letter and number describing the respective brain region and brain side [15]. If using traditional wet electrodes, the skin must be cleaned, and conductive paste must be applied during the electrode application, in order to lower the electrode-skin impedance. On the other hand, dry electrodes can be applied directly to the skin, greatly reducing the required preparation time. However, these electrodes have larger and more unstable electrode-skin impedances, reducing the quality of the recorded data [16].

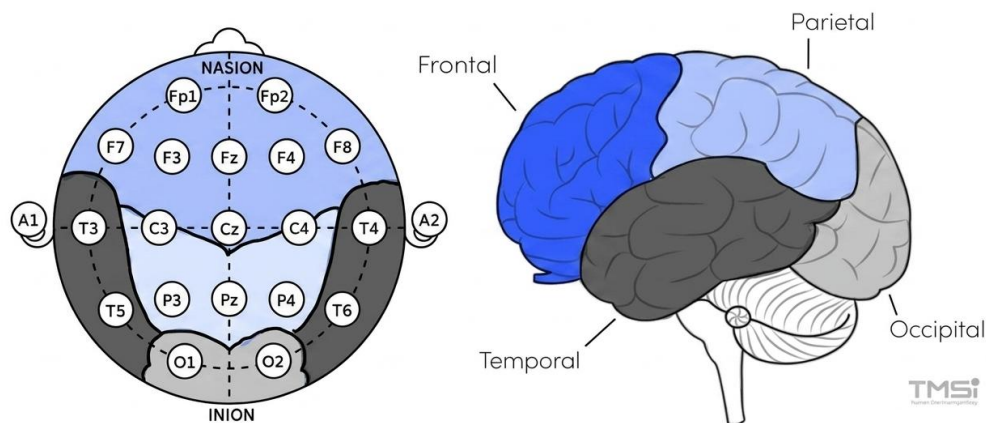


Figure 2: Diagram showing 10-20 System Electrode Locations [17]

When the electrode input reaches the EEG machine, it is first amplified by an instrumentation amplifier to increase its magnitude and remove common-mode noise. The amplifier should have a common-mode rejection ratio (CMRR) of at least 100 dB to sufficiently remove mains frequency interference [18, p. 1365]. The signal is then bandlimited by an analogue filter, in order to prevent aliasing during the analogue to digital conversion. Finally, an analogue to digital converter (ADC) digitises the EEG signal so it can be processed by a microcontroller [18, p. 977]. EEG headsets typically utilise delta-sigma

ADCs, as their high resolution reduces the need for heavy amplification and filtering, reducing the number of required discrete components [19]. A simple block diagram for an EEG system is shown in Figure 3.

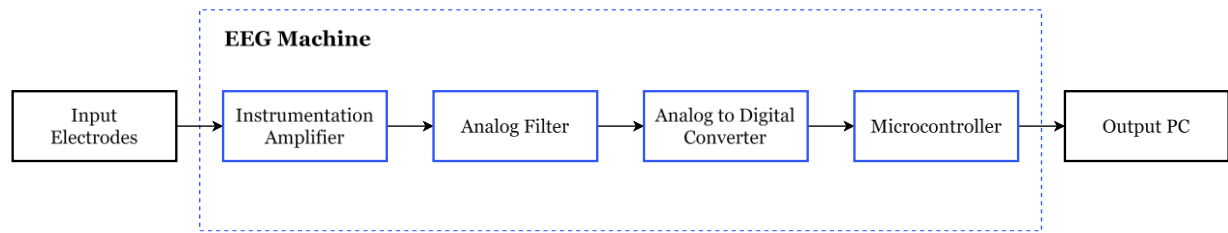


Figure 3: EEG system block diagram

Once the EEG signal has been digitised by the ADC, it must be transmitted to a computer for signal processing, visualisation, and data analysis. It is important that the data has accurate time synchronisation in order to extract time locked potentials that have been induced by external stimuli. To solve this problem, a framework called The Lab Streaming Layer (LSL) is often used to synchronise data streams from different devices, whilst accounting for network delays, jitters, disconnections and offsets [20]. As well as providing their own software interfaces, companies like OpenBCI and Emotiv both natively support LSL streaming [21], whilst for other platforms, like the Muse, have community created LSL platforms. Once data has been pushed to the Lab Streaming Layer, it can be recorded or visualised by a host of open-source tools, examples of which are given in Appendix I.

EEG machines can be used to record spontaneous EEG activity, as well as event related potentials (ERPs). Spontaneous EEG activity is the background activity of the brain, occurring without any specific external stimulus. It is categorised into frequency bands, each loosely representing different levels of cerebral activity [22]. For example, alpha waves have a frequency of 8 to 13 Hz, and are typically measured from the occipital region of the cortex, from an awake patient with their eyes closed [1, p. 21].

On the other hand, ERPs are time locked EEG responses. These can either be exogenous, commonly referred to as evoked potentials (EPs), which are generated as a result of physical stimulation by a sensory stimulus, or they can be endogenous, being generated by internal cognitive and motor processes [23]. For instance, when a patient observes a light source flickering at a certain frequency, potentials of the same frequency can be measured from the occipital areas of the brain [24]. These are called steady state visually evoked potentials (SSVEPs). On the other hand, possibly the most widely studied endogenous ERP is the P300; a broad positive waveform occurring approximately 300 ms after a patient receives external stimulation [1, p. 62], thought to originate from the brain processing information related to attentional and memory mechanisms [25].

3. LITERATURE REVIEW

Prior to starting development, a review of existing commercial EEG platforms and relevant academic literature was conducted to gain insight into gaps in the EEG market, to find opportunities for improvement in the current lineup of technology, as well as to inform the design choices made throughout the project.

3.1. Commercial EEG Modules

Out of the several commercial EEG headsets on the market, headsets produced by Emotiv have been the most widely used in scientific research [4]. Their EPOC X headset shown in Figure 4.1, provides a highly integrated, user-friendly 14-channel system, albeit with varying levels of reported accuracy [5] [26] [27]. However, its restrictive pricing model limits its ability to be used in an educational environment. The EPOC X is currently priced at \$999 [8], while the 32-channel Mind Flex 2 starts at \$1,899 [28]. Moreover, accessing and exporting raw EEG data requires a subscription to their EmotivPRO software, costing \$149 per month for a standard tier subscription [6].

In comparison, OpenBCI produce a fully open-source EEG platform, which gives the user full control over the entire EEG process. Their Cyton PCB module, shown in Figure 4.2, lets the user choose the number, type and location of electrodes, as well as whether the data is viewed on the OpenBCI GUI, or pushed straight to the Lab Streaming Layer. Whilst providing a platform better suited for research, and of comparable quality to clinical EEG machines [29], the hardware costs remain high. The Cyton board alone costs £933, whilst the complete Ultracortex, which integrates the Cyton board with their own 3D-printed headset and dry-electrodes, costs £2,255 [30].

More recently the Cerelog ESP-EEG board, shown in Figure 4.3, was introduced to the market [31], also with the intention of lowering the cost of BCI technology. Similarly to the OpenBCI Cyton, the Cerelog module is built around the Texas Instruments (TI) ADS1299 [32], which integrates EEG amplification, analogue to digital conversion, as well as other EEG specific features, to provide a fully integrated EEG analogue front end [14]. The Cerelog platform also has strong software integration, allowing the user to view their data on a fork of the OpenBCI GUI, or use the popular Brainflow EEG library to access raw EEG data. However, the system does not yet support wireless data transfer, relying instead on a USB connection. As well as restricting patient mobility, this presents a danger to the user if the host computer is connected to mains, as there will be no electrical isolation of the patient. Furthermore, the hardware remains relatively expensive, with a base price of \$650.

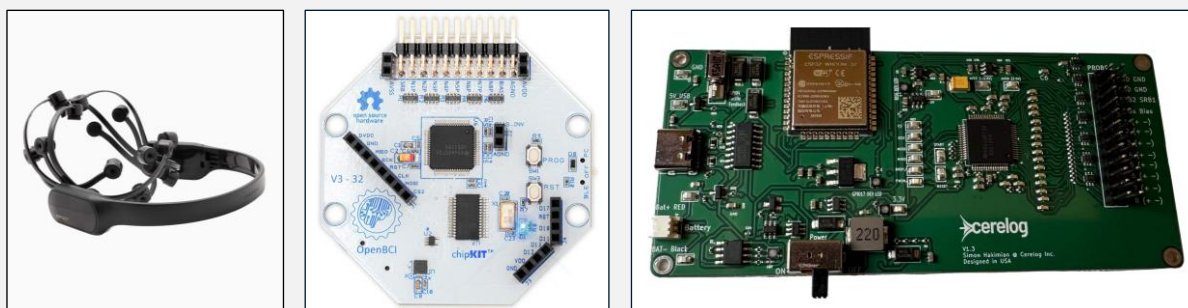


Figure 4: From left to right: Emotiv EPOC X [8], OpenBCI Cyton [33], Cerelog ESP-EEG [31]

3.2. Academic Low-Cost Prototypes

As well as commercial offerings, there are also several studies that document the production of low-cost EEG platforms.

In 2013, Pennsylvania State University researchers also identified a need for low-cost EEG modules to use as an educational tool [9]. They observed difficulties in getting student buy-in for their BCI classes, as the high hardware cost meant students had limited access to EEG equipment. An older, electrocardiogram (ECG) specific version of the aforementioned TI chip; the ADS1298, was paired with a TI MSP430 series microcontroller, known for its exceptionally low power consumption [34], to create an EEG acquisition module with a component cost of under \$100, shown in Figure 5. They then validated their system through the detection of alpha waves and a visually evoked p300 response. Overall, a strong EEG platform has been presented, with minor improvements being wireless data transfer, battery power, and stronger software integration.



Figure 5: Pennsylvania State University EEG Module

In 2017 Nguyen et al. [35] documented an ADS1299 based EEG system, this time using an STM32F4 microcontroller, which they validated using alpha wave detection and Electrooculography (EOG) eye tracking tasks. To overcome the mobility limitations of previous wired systems, Bluetooth data transmission was implemented using a HC – 05 Bluetooth module. However the authors admit the system was susceptible to high levels of noise, possibly due to not utilising the ADS1299's driven right leg functionality, highlighting the importance of using the patient bias as an active ground to cancel common mode noise.

More recently, Gao (2023) [36] developed another ADS1299 STM32 EEG system, with the intent of creating an EEG module possessing the accuracy of large laboratory EEG setups, but on a much smaller scale. Once again a separate bluetooth module, the NRF52832, was used to send data to a computer. The system, shown in Figure 6, was verified by using fourier analysis to test for SSVEPs across multiple stimulus frequencies. This paper goes further than the previous examples in documenting their work, with snapshots of electrical schematics, however it is still far from being easily reproducible. Additionally the system is not integrated with any established EEG software frameworks, like brainflow or LSL.

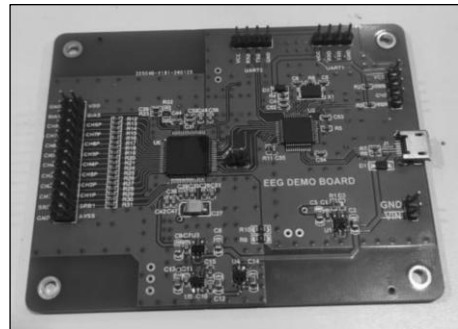


Figure 6: Gao (2023) EEG Module

In conclusion, the ADS1299 has established itself as the analog front end of choice in low-cost, portable EEG modules, both in commercial products and in academic research. Additionally, alpha wave, P300, and SSVEP detection methods have repeatedly been used as methods of verifying the accuracy of proposed systems. Overall strong proofs of concept for small scale, low cost ADS1299 based EEG systems have been provided. However, the work done in these papers is not easily reproducible, and despite the decreasing costs of EEG hardware, users and researchers are still left without a truly low-cost solution.

4. SYSTEM DESIGN

4.1. Component Selection & System Architecture

4.1.1. ADC Selection

The first stage of the design process involved selecting a suitable analogue to digital converter, as this component dictates the overall system architecture.

As evidenced by the literature review, the ADS1299 is the standard for low-cost EEG modules. This is primarily due to its exceptionally low input referred noise, high CMRR, and large input impedance [14]. The ADS1299 also comes equipped with several EEG specific features, including lead-off detection to identify poor electrode-skin contact, built in driven right leg circuitry for common mode noise cancelation, and an internal multiplexer which allows for the routing of the reference electrode to the negative pin of each differential input. However, this high level of integration carries a financial premium, as the ADS1299 currently retails for £64.57 from Mouser Electronics. To adhere to the low-cost objective of the project, cheaper alternatives were evaluated.

One alternative approach is to use an ECG analogue front end, similarly to how the ADS1298 was utilised at Pennsylvania University. However, since these ICs are intended for ECG applications, where signal amplitudes are orders of magnitude larger [37], they offer comparatively poor noise performance. Another option is to use a non-application specific ADC, such as the Analog Devices AD7779, that also features 8 simultaneously sampled 24-bit channels.

Table 2 compares the performance metrics of these ADCs, specifically evaluating their theoretical capability to measure a 10 μV_{pp} P300 ERP [38]. Referred to input (RTI) noise levels are assuming a maximum PGA gain and minimum data rate, and thus represent a lower bound. The RTI least significant bit (LSB) size has been calculated with Equation 1:

$$LSB_{RTI} = \frac{2 \cdot \left(\frac{V_{ref}}{PGA \text{ Gain}} \right)}{2^{Bits}} \quad (\text{Equation 1})$$

Table 2: ADC performance comparison when measuring a 10 μV_{pp} P300 [14], [39], [40], [41]

IC	RTI Noise (μV_{pp})	P300 Noise Contamination (%)	PGA Gain	RTI LSB (μV)	P300 Quantisation Steps	Mouser Cost
ADS1299	0.98	9.8	24	0.022	447.39	£64.57
ADS1298	2.7	27	12	0.040	251.66	£34.39
AD7779	2.28	22.8	8	0.037	268.44	£14.49
AD8422*	0.15	1.5				£5.60

*Note: The AD8422 is an instrumentation amplifier, not an ADC.

The ADS1299 has the lowest RTI noise, as well as the lowest RTI LSB size, enabling precise and low noise acquisition of a P300 wave. In contrast, using the ADS1298 or AD7779 would result in a P300 signal that is contaminated with approximately 25% noise, and recorded at almost half the effective resolution.

The lower performance metrics of the cheaper ADCs could be compensated for by placing an instrumentation amplifier stage prior to the ADC, for instance the AD8422, which has an

ultra-low RTI noise of $0.15 \mu\text{V}_{\text{pp}}$. However, despite outperforming the ADS1299 in terms of noise performance, this would add increased complexity and cost, since each channel would require a dedicated instrumentation amplifier.

Consequently, the ADS1299 was selected. Its low RTI noise and high PGA gain ensure high fidelity signal acquisition, while its EEG specific features minimise circuit complexity and reduce the need for other components. Moreover, in some cases the increased cost can be mitigated by using the 6-channel, or 4-channel variants, which are priced at £51.52 and £32.75 respectively.

4.1.2. Microcontroller Selection

Following the selection of the analogue front end, a suitable microcontroller was required to manage SPI (Serial Peripheral Interface) communication with the ADS1299, and send the EEG data to a host computer.

Project Objective 2 states that data transmission from the EEG module to a host computer must be completely wireless. This is primarily to ensure electrical isolation between the user and mains power; however, it also removes the physical tether to the host device, granting the user full mobility. Consequently, the search was restricted to microcontrollers with Bluetooth integration. Utilising an external Bluetooth module would add unnecessary complexity to the project by way of introducing another serial bridge to the data path, not to mention increasing the bill of materials (BOM), and consuming additional PCB real estate.

This primary constraint was used to narrow down the field to three primary candidates, summarised in Table 3. The STM32 architecture was considered due to its high performance, low power consumption and robust ecosystem [42]. While the STM32F4 range of microcontrollers used by Nguyen et al. [35] and Gao [36] do not come with Bluetooth, STMicroelectronics have a capable dual core Bluetooth offering in the STM32WB55. Alternatively, one of the most widely used microcontrollers for Bluetooth Low Energy (BLE) applications is the Nordic Semiconductor nRF52840, due to its low power consumption and an extremely stable Bluetooth stack. In fact Nordic sells approximately 40% of all BLE chips in industry [43].

Table 3: Bluetooth microcontroller comparison

Micro	Cores	Clock Speed	Max RAM	Max Flash	Unit Cost
STM32WB55	1x ARM Cortex-M4 1x ARM Cortex-M0+	Core 1: 64 MHz Core 2: 32 MHz	256 KB	1 MB	£6.77
nRF52840	ARM Cortex-M4	64 MHz	256 KB	1 MB	£4.85
ESP32-S3	2x Xtensa LX7	240 MHz	512 KB	16 MB (external)	£1.39

Despite the popular STMicroelectronics and Nordic offerings, the Espressif ESP32-S3 was chosen for this project. As evidenced by Table 3, the ESP32-S3 outperforms the STM32WB55 and nRF52840 in terms of computational power, possessing two 240 MHz cores, and double the amount of SRAM. This dual core architecture allows for data reading, and data sending to each be localised to an individual core, in a way that would not be possible on the nRF52840.

From a hardware design perspective, the ESP32-S3 is widely available as a pre-certified, shielded module: the ESP32-S3-Wroom-1, containing an integrated PCB antenna, crystal,

and external flash memory. This, combined with native USB support, once again removes unnecessary complexity from the project, for example RF layout and USB to UART conversion. Additionally, the module form factor is priced very reasonably, with costs starting at £3.80, and development boards costing around £10.

4.1.3. Electrode Selection

While the EEG module is intended to be electrode agnostic, a set of electrodes needed to be selected for system validation. Electrodes generally differ across two primary metrics: active vs passive, and wet vs dry.

Active electrodes, shown in Figure 7.1, amplify the EEG signal directly at the electrode, preventing undesired electrical noise, particularly at mains frequency, from coupling into the lead wires and contaminating the signal. These would require dedicated power routed from the PCB, and are typically more expensive, and more difficult to source than their passive counterparts, that simply transmit the signal to the EEG machine for amplification. Furthermore, research indicates that in high quality, laboratory conditions, specifically with low electrode-skin impedances, passive electrodes provided a cleaner baseline signal [44].



Figure 7: EEG electrode types: active electrodes (left) [45], dry electrodes (middle) [46], and passive gold cup electrodes (right) [47].

Regarding electrode application, wet electrodes are the clinical standard. These require the use of a conductive paste, gel or saline solution during the application process, which will dry out after a certain amount of time. Moreover, the skin needs to be prepared before application to reduce the electrode skin impedance, typically by applying an abrasive gel like Nuprep or using alcohol wipes and mechanical abrasion. On the other hand, dry electrodes, as shown in Figure 7.2, can be applied directly to the scalp without the need for preparation. While convenient, this comes at a cost to performance, as they were found to produce significantly larger noise levels compared to both active and passive wet electrodes when conducting a P300 oddball test [48].

For this project, passive wet gold cup electrodes were chosen, primarily due to their low cost, wide availability, and reliable signal acquisition. While the ease of use of dry electrodes is well suited to an educational environment, establishing clean EEG data was prioritised for the initial hardware tests. However, in the future it may be worthwhile testing the system with dry electrodes, to assess their viability. A total of eight electrodes were sourced from Chinese app Pinduoduo, amounting to £15.

4.1.4. Overall System Architecture

Following the selection of the primary hardware components, a complete system architecture was defined. Figure 8 illustrates this architecture, mapping out the signal flow from the brain to the final GUI.

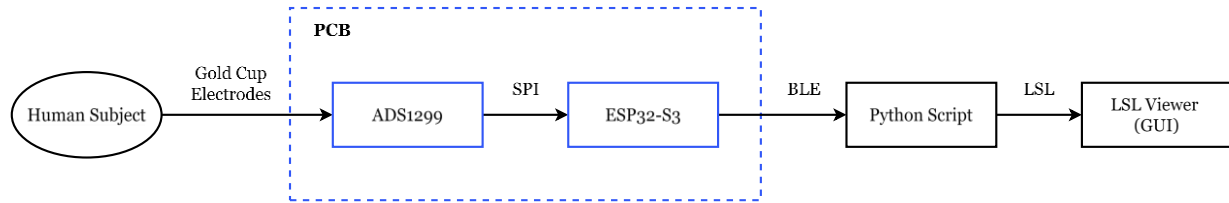


Figure 8: System architecture flow chart

EEG data is first acquired from the scalp using gold cup electrodes. This necessitates skin preparation as well as the use of conductive paste during the application process. The ADS1299 then amplifies the signal, removing common mode noise, before digitising the data from a maximum of 8 channels.

The digitised data is subsequently transmitted via SPI to the ESP32-S3 microcontroller, where it is batched into packets to be transmitted using the BLE protocol, with the ESP32-S3 acting as a GATT (Generic ATtribute Profile) server. This ensures that electrical isolation is maintained between the user and mains.

Finally, at the host computer, a python script will act as the GATT client, subscribing to notifications from the GATT server. The script unpacks the incoming byte arrays, including checking for lost packets, before pushing the data to an LSL outlet. Utilising the LSL framework allows the data to be viewed in real time using tools such as BrainVision LSL Viewer or OpenViBE, and recorded using Lab Recorder. Additionally, it allows users to synchronise the recorded data with external stimuli, through the use of LSL marker streams.

4.2. Bench Prototyping and Firmware Design

4.2.1. Initial Prototyping

Once component selection had been completed, the development boards for the ADS1299 (ADS1299EEGFE-PDK) and ESP32-S3-Wroom-1 (S3-DevKitC-1) were purchased to conduct initial tests.

To establish a performance baseline prior to introducing custom firmware, the ADS1299 was first evaluated using the included MMBo motherboard. This board interfaces between the ADS1299 and the Texas Instruments evaluation software, allowing for visualisation and Fast Fourier Transform of channel inputs, and direct manipulation of the internal control registers.

Initial validation was conducted by connecting the positive channel and reference inputs of the development board to a signal generator. Figure 9.1 shows the results of a 1 mVpp input sine wave, while Figure 9.2 shows the respective frequency domain FFT. Further validation was performed by routing the ADS1299's internally generated 3.6 mVpp test signal to channel 1, the results of which are shown in Figure 9.3.

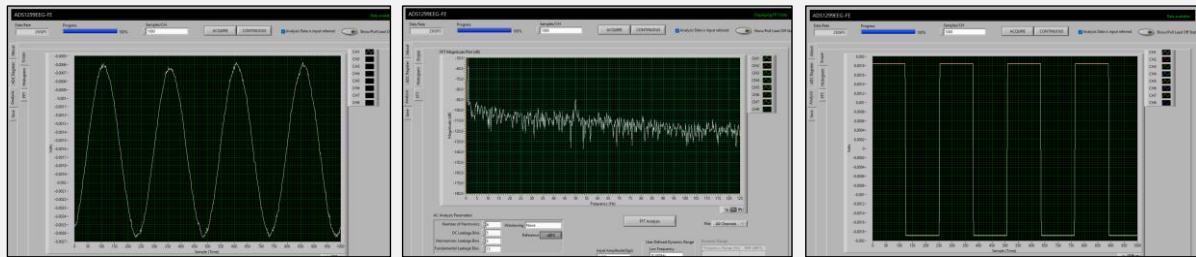


Figure 9: ADS1299 EVM tests: 1 mVpp input (left), 1 mVpp FFT (middle), internally generated square wave (right)

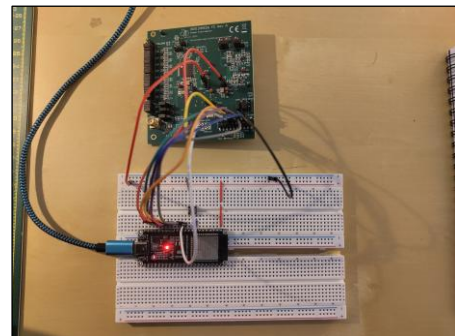
Further bench tests verified the DRL circuitry, PGA configuration and bipolar supply operation. In addition, the intrinsic noise floor was recorded at 2 μ Vpp, confirming the device was operating as set out in the datasheet.

This initial testing phase was useful for understanding functionality and register setup of the ADS1299. Once completed, the analogue front-end board was separated from the MMBo motherboard, and jumper wired to the S3-DevKitC-1, as shown in Figure 10. Table 4 lists the wire connections between the two boards.

Table 4: ADS1299 ESP32 communication pins

ESP32 Pin	ADS1299 Pin
8	RESET
9	DRDY
10	CS
11	DIN (MOSI)
12	SCK
13	DOUT (MISO)
18	START

Figure 10: Development Setup



4.2.2. ADS1299 Communication Library

The next stage of development involved the creation of a library to interface the ESP32-S3 with the ADS1299, which communicates using SPI. Aside from the four main SPI pins; DIN, DOUT, SCK, and CS, the ADS1299 also has several dedicated communication pins including RESET, START, PWDN and DRDY.

During standard operation the ADS1299 is placed in the Read Data Continuous (RDATA) mode. When the START pin is low, and on the falling edge of the DRDY pin, DOUT releases 216 bits of data, comprising of 24 status bits, followed by eight 24-bit channel readings. In order to configure the device, including reading from and writing to internal registers, the data stream must be halted using the Stop Read Data Continuous Command (SDATA), releasing the SPI buses.

An open-source communication library for the ADS1299 already existed, having been developed by Conor Russomanno, one of the founders of OpenBCI [49]. However, this library was specifically designed for 8-bit AVR microcontrollers, for instance the ATmega328P found on the Arduino Uno R3, as it manipulates the AVR specific SPI control registers directly. This renders the library incompatible with the 32-bit Xtensa processors found on the ESP32-S3. Furthermore, the original library was designed to act as a pass through, printing all received data straight to the serial port to be processed by a host PC, while the architecture used in this project transmits data over Bluetooth instead.

To resolve these incompatibilities, Russomanno's library was modified to use the standard Arduino SPI library, making it compatible with any microcontroller using the Arduino framework. Additionally, the following changes were implemented:

1. Rather than pushing recorded bits straight to the serial monitor, they are sign extended into 32-bit integer longs, and which are then returned as outputs.
2. The `updateData` function responsible for reading data from the ADS1299 was changed with the intention of being called by an interrupt when the DRDY pin has been pulled low, rather than being polled from the main loop.
3. A new `updateDataPacket` function was added, which skips sign extension and reading the data into a long, instead dropping the channel data directly into a byte array, to be sent over Bluetooth.
4. Correct power up sequencing was added to the setup function, removing the need for the user to manually follow the power-up sequence set out by the datasheet before sending instructions.

The complete source code for the revised ADS1299 communication library can be found in Appendix B.

4.2.3. ESP32-S3 Firmware

With the ADS1299 communication library complete, the ESP32-S3 firmware was developed with Platformio, using the Espressif's Arduino Core. The firmware has two primary objectives: to accurately read data from the ADS1299 on the falling edge of the DRDY pin, and to transmit data to the host computer using Bluetooth Low Energy, with high-throughput and no data loss. Combining these two objectives onto a single execution thread risks one task blocking the other, particularly due to the precise timing requirements of each task. Even at the lowest data rate, the ADS1299 is sending a 216-bit sample every 4 milliseconds, that needs to be read, decoded, packaged and transmitted. To solve this problem, the dual core architecture of the ESP32-S3 was utilised, with one core in charge of reading data, and the other responsible for transmitting it. The overall structure of the firmware can be seen in Figure 11.

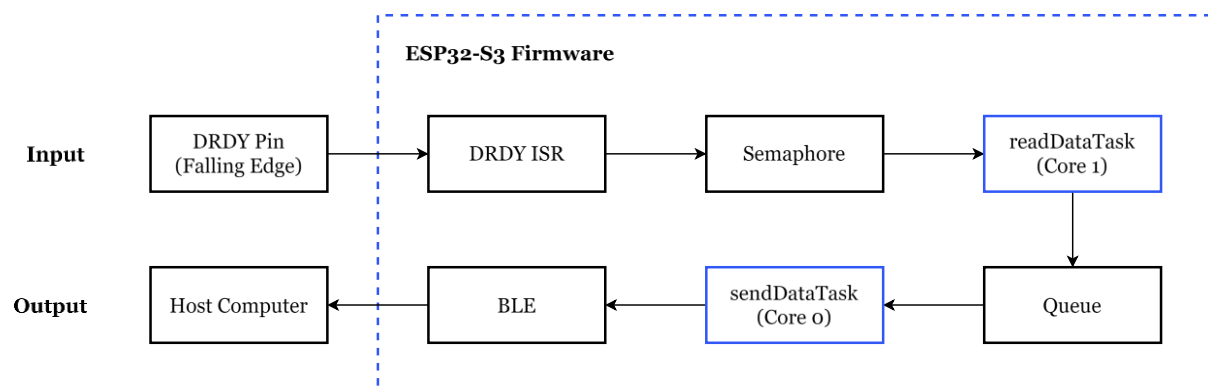


Figure 11: ESP32-S3 firmware architecture diagram, showing dual core pipeline.

When the DRDY pin is pulled low, indicating that there is a new sample available, an interrupt service routine (ISR) is called, which gives to a binary semaphore, and immediately yields to a higher priority task, if one is waiting. The ISR, shown in Listing 1, has been kept minimal, to ensure that other DRDY interrupts are not delayed, reducing the risk of sample loss.

Listing 1: DRDY ISR function

```

void IRAM_ATTR dataReadyISR() {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    xSemaphoreGiveFromISR(drdySemaphore, &xHigherPriorityTaskWoken);
    if (xHigherPriorityTaskWoken) {
        portYIELD_FROM_ISR();
    }
}

```

The task of reading data from the ADS1299 is pinned to core 1 of the ESP32-S3, at a priority level of 3. This task, shown in Listing 2, spends most of its time blocked, waiting for the semaphore to be released. When the semaphore is given by the ISR, the task wakes up and increments a sample counter, which is put in byte 0 of a 28-byte packet. The updateDataPacket function is then called to fill the remaining 27 bytes with EEG data read from the ADS1299, and the completed packet is pushed to a queue, to be sent to the secondary task running on the other core. The sample counter can be used by the host to determine if any samples have been lost.

Listing 2: Read data task (core 1)

```

...
for(;;) {

```

```

if (xSemaphoreTake(drdySemaphore, portMAX_DELAY) == pdTRUE) {
    dataPacket[0] = sampleCounter++; // 1. Assign the Sample Counter
    ADS.updateDataPacket(dataPacket); // 2. Read the SPI Data
    xQueueSend(packetQueue, dataPacket, 0); // 3. Send to the Bluetooth Core
}
}
...

```

The task of sending data to the host computer runs on Core 0 at a priority level of 1, alongside the BLE stack, and is shown in Listing 3. This core is blocked until a sample becomes available in the queue, at which point it is added to a 224-byte batch buffer. Once eight packets have been received, and the buffer is full, the batch is transmitted as a single GATT notification. Batching the data is necessary as BLE notifications carry overheads, and thus sending each 28-byte sample individually would consume a greater amount of radio time. In particular a factor of 8 was chosen as it fits nicely within BLE's 251-byte extended data channel payload [50], maximising the data sent with each notification.

Listing 3: Send data task (core 0)

```

...
// Wait for a single 28-byte packet from Core 1
if (xQueueReceive(packetQueue, &singlePacket, portMAX_DELAY) == pdPASS) {
    // Copy the 28 bytes the 224-byte batch buffer
    memcpy(&batchBuffer[batchCount * 28], singlePacket, 28);
    batchCount++;
    // After 8 packets, send the full 224-byte batch
    if (batchCount == 8) {
        pCharacteristic->setValue(batchBuffer, 224);
        pCharacteristic->notify();
        batchCount = 0; // Reset the batch count
    }
}
...

```

The sample queue has a depth of 256, providing approximately half a second of buffering when the ADS1299 is running at 500 SPS, absorbing any short term BLE delays. The system is capable of transmitting data at 500 SPS with no data loss (equivalent to 14 kB/s), however at 1000 SPS packets start to be dropped due to limitations regarding the windows connection interval. A possible mitigation would have been to use Bluetooth Classic rather than BLE, which provides a continuous stream of data, but is only supported on the original ESP32, and not the newer ESP32-S3 selected for this project. Nevertheless, 500 SPS, which results in a Nyquist frequency of 250 Hz, is more than enough for most EEG applications, where the data is often filtered to cutoff frequencies far below this value.

The code is also written such that there is an initial connection phase, where the ADS1299's DC lead off circuitry is active. This is where a small excitation current is injected to each of the active electrodes, and a comparator threshold detects whether the resulting voltage indicates an open circuit. Once the connections are verified, the user can press the boot button to disable the lead-off circuitry, which would have otherwise contaminated the EEG measurements.

The completed firmware uses 900 KB of flash, and 140 KB of internal SRAM, leaving 7.1 MB and 248 KB respectively free for future development. The complete source code for the ESP32-S3 can be found in Appendix C.

4.3. Host Software Design

4.3.1. Python BLE Client and LSL Bridge

The host software is responsible for receiving and decoding the EEG data stream sent by the ESP32-S3, before pushing the data to the Lab Streaming Layer for visualisation and recording. Python was chosen as the implementation language due to the availability of mature libraries for relevant tasks: Bleak for receiving BLE data, Pylsl for pushing data to the LSL, and Asyncio to run and manage an event loop.

Python's Asyncio framework was used to manage the operation of the host script. Asyncio implements cooperative concurrency on a single thread, which is controlled by a central event loop, scheduling tasks that voluntarily yield control at explicit 'await' points. Unlike multi-threading, this model guarantees that no task is interrupted mid execution, eliminating the need for locking of variables to prevent race conditions. The Bleak library by default runs on top of Asyncio, as BLE communications are inherently I/O bound, and the system needs to be able to read the notification data without blocking other sections of the code. The script is organised into three concurrent tasks running on the event loop, and the overall structure can be seen in Figure 12.

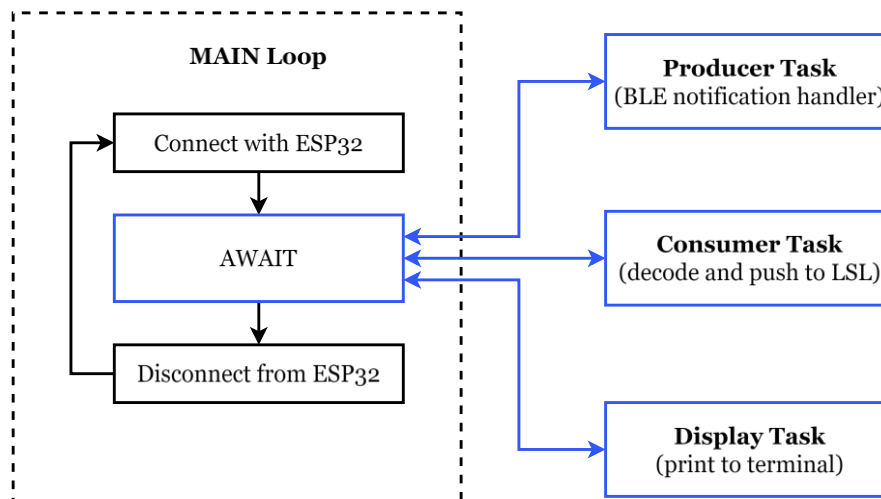


Figure 12: Python script structure diagram

The producer task is the notification handler, that when a BLE notification arrives, performs a single length check, before depositing the 224-byte batch into a queue. Similarly to the DRDY ISR, the notification handler returns as quickly as possible to avoid blocking subsequent data packets.

The consumer task awaits batches from the queue, unpacking each 224-byte batch into its eight constituent ADS1299 samples. The first byte of each sample contains the sample counter assigned by the ESP32-S3 (Listing 2), bytes 1-3 contain the ADS1299 status bits, including the results of lead-off detection, while the remaining 24 bytes contain the eight channel readings. Upon dividing the batch, drop detection is performed by tracking the expected next sample counter. If the counter has advanced by more than one, a counter tracking the number of dropped samples is incremented, and a NaN filled sample is pushed to the LSL, in order to maintain temporal alignment. This process is shown in Listing 4.

Listing 4: Consumer task drop detection

```
...
if state.expected_counter is not None:
    dropped = (sample_counter - state.expected_counter) % 256
    if dropped > 0:
        state.drop_counter += dropped
        # Push what we have so far, then insert NaNs, then continue
        if chunk:
            outlet.push_chunk(chunk)
            chunk = []
        nan_sample = [math.nan] * 8
        outlet.push_chunk([nan_sample] * dropped)
state.expected_counter = (sample_counter + 1) % 256
...
```

The consumer task then decodes the data for each channel, by reassembling its three constituent bytes into a 24-bit unsigned integer, sign extending to 32-bits, and multiplying by the scale factor shown in Equation 2 to obtain the channel reading in microvolts. Decoded samples are accumulated into a chunk before being pushed to the LSL outlet, reducing the number of times the push_chunk() function has to be called.

$$V_{ch} = raw \cdot \frac{2 \cdot V_{ref}}{Gain \cdot 2^{24}} \quad (\text{Equation 2})$$

A final display task runs every 100 ms, printing the most recently received sample, the dropped samples counter, the current queue depth, and the lead-off status bits, allowing the user to verify that data is flowing correctly, without having to launch a separate viewer. Rather than reading from the queue directly, the task reads from a shared state that was populated by the consumer task.

The main event loop is responsible for device discovery, and connection and reconnection. On startup, the script scans for an advertising peripheral with device name: ESP32_EEG_DATA, and then subscribes to its notifications. If the disconnection event is triggered by Bleak's disconnection callback, the loop resets and begins scanning for the ESP32 again. On the other hand, if the stop event is triggered by the user pressing CTRL+C, the loop is terminated, and the program ends.

The complete source code for the host script can be found in Appendix D. This code was packaged into a 12 MB executable file using PyInstaller.

4.4. PCB Design

4.4.1. Electrical Schematic

With the firmware and host software validated using the ADS1299 evaluation module and ESP32-S3 devkit, the next stage of development was to integrate the prototype into a single PCB. The principal goal when designing the PCB was keeping noise and potential interference around the ADS1299 to a minimum to guarantee high quality EEG measurements. The PCB also needs to be battery powered, in order to maintain electrical isolation, as well as being able to be connected to a computer for flashing. The schematic and layout were developed using Altium Designer, and the OpenBCI Cyton schematic [51] was used as a reference for some of the analogue front end circuitry. The complete schematic can be viewed in Appendix E.

Most EEG electrode sets come with touchproof (DIN 42802) connectors, as these are the standard for connecting to clinical EEG machines. However sourcing PCB mount, touchproof connector receptacles proved a challenge, as these are components typically only sold to suppliers of medical equipment. Therefore standard 2.54 mm pin headers were used as the electrode inputs, due to the fact that some electrode sets can be bought with Dupont style terminations. Alternatively, the touchproof connectors can quite easily be removed and replaced by soldering female Dupont wires, which is the method that was used for this project.

The electrode inputs are then routed to bi-directional ESD protection diodes, that prevent voltage spikes originating from the body from damaging the ADS1299. Additionally, the electrode inputs pass through an RC low-pass filter before reaching the ADS1299 channel inputs. This is primarily to attenuate modulation frequency noise (1.024 MHz), as the digital decimation filter inside the ADS1299 has unity gain at multiples of f_{MOD} , and thus any noise at these frequencies is passed straight to the ADC inputs. The filter is comprised of a 2.2 k Ω resistor, and a 1 nF capacitor, resulting in -23 dB gain at 1.024 MHz. Class 1 capacitors were used here rather than the standard Class 2 for their improved overall stability, with CoG having a $\pm 0.3\%$ capacitance change across its operating temperature, compared to $\pm 15\%$ for X7R [52]. Class 2 capacitors also exhibit piezoelectric behaviour that can create electrical noise [53].

It was chosen to operate the ADS1299 in bipolar supply mode, so that the electrode inputs are naturally centred around 0V, rather than 2.5V. Table 5 shows the required supply rails for the PCB.

Table 5: PCB supply rail characteristics

Device	Supply Rail	Supply Voltage	Typical Current	Peak Current
ADS1299	AVDD	2.5V	7.14 mA	
	AVSS	-2.5V	- 7.14 mA	
	DVDD	3.3V	1 mA	
ESP32-S3	DVDD	3.3V		202 mA

A 3.7V, 2000 mAH LiPo battery was chosen to power the PCB, as the nominal voltage can be easily stepped down to 3.3V, using a low noise, low-dropout regulator (LDO), with an efficiency of 89%. This is ideal, as using switch mode power supplies would introduce voltage ripple and electromagnetic interference (EMI) [54], which could contaminate the EEG signals. Furthermore, assuming a total current draw of 216.28 mA, and approximately 80% overall efficiency, the battery should be able to power the system for a minimum of 7.40 hours.

Figure 13 shows the PCB power supply schematic. AVDD and AVSS for the ADS1299 were generated using a single integrated charge pump LDO, the LM27762, specifically designed for low noise applications. Meanwhile the 3.3V digital supplies for the ADS1299 and the ESP32-S3 were generated with separate low noise LDOs, in order to prevent load induced output droop and voltage ripple caused by the ESP32 from coupling into the ADS1299.

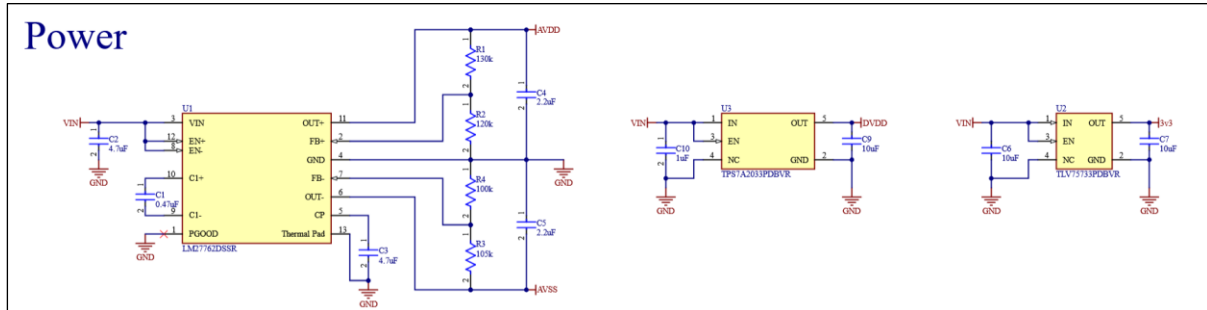


Figure 13: PCB power supply schematic

The ADS1299 and ESP32 schematics were created adhering as closely as possible to the datasheet recommendations, and following the examples set out by their respective development boards. The ESP32 is flashed using its native USB, and reset and boot buttons have been included in the schematic to allow the ESP32 to be put into programming mode. A SPDT switch on the board allows the user to select either USB 5V, or the LiPo battery 3.7V to be used as the input power source, which can also act as an on-off switch when the USB is not connected. ESD protection diodes have been placed on the USB input, as well as connected to the input battery JST-PH, as these are components that will be touched frequently.

Finally, UART GPIO pins of the ESP32 have been broken out onto pin headers, to act as a failsafe in case the USB communication were to fail, and also to be used as spare GPIO, for instance if the user wishes to control an external stimulus using the ESP32. Test points have also been placed on the board for each of the supply rails, to facilitate easy troubleshooting in the event of a problem.

4.4.2. PCB Layout

Once again, when laying out the PCB, layout recommendations from component datasheets were adhered to as closely as possible. A four layer; signal, ground, power, signal stackup was used to simplify routing, but most importantly to reduce potential noise. Using a ground plane means that all signals have low-impedance return paths, keeping current loop areas small and reducing cross talk. Furthermore, a ground plane can act as a Faraday cage, absorbing interference from nearby circuits, resulting in stronger signal integrity [55].

The ADS1299 datasheet recommends partitioning the board into analogue and digital sections, as is standard in mixed signal design, in order to prevent digital noise coupling into analogue signals. This approach negates the need for a split ground plane, as digital return currents will remain under the digital signal traces [56, p. 663]. Figure 14 shows the placement of the analogue electrode inputs and ADS1299 on the left side of the board, while the digital ESP32-S3 and power generation circuitry has been placed on the right. Components have been placed predominantly on the top layer for ease of assembly, and the ESP32-S3 antenna has been hung off the edge of the board, to ensure proper clearance.

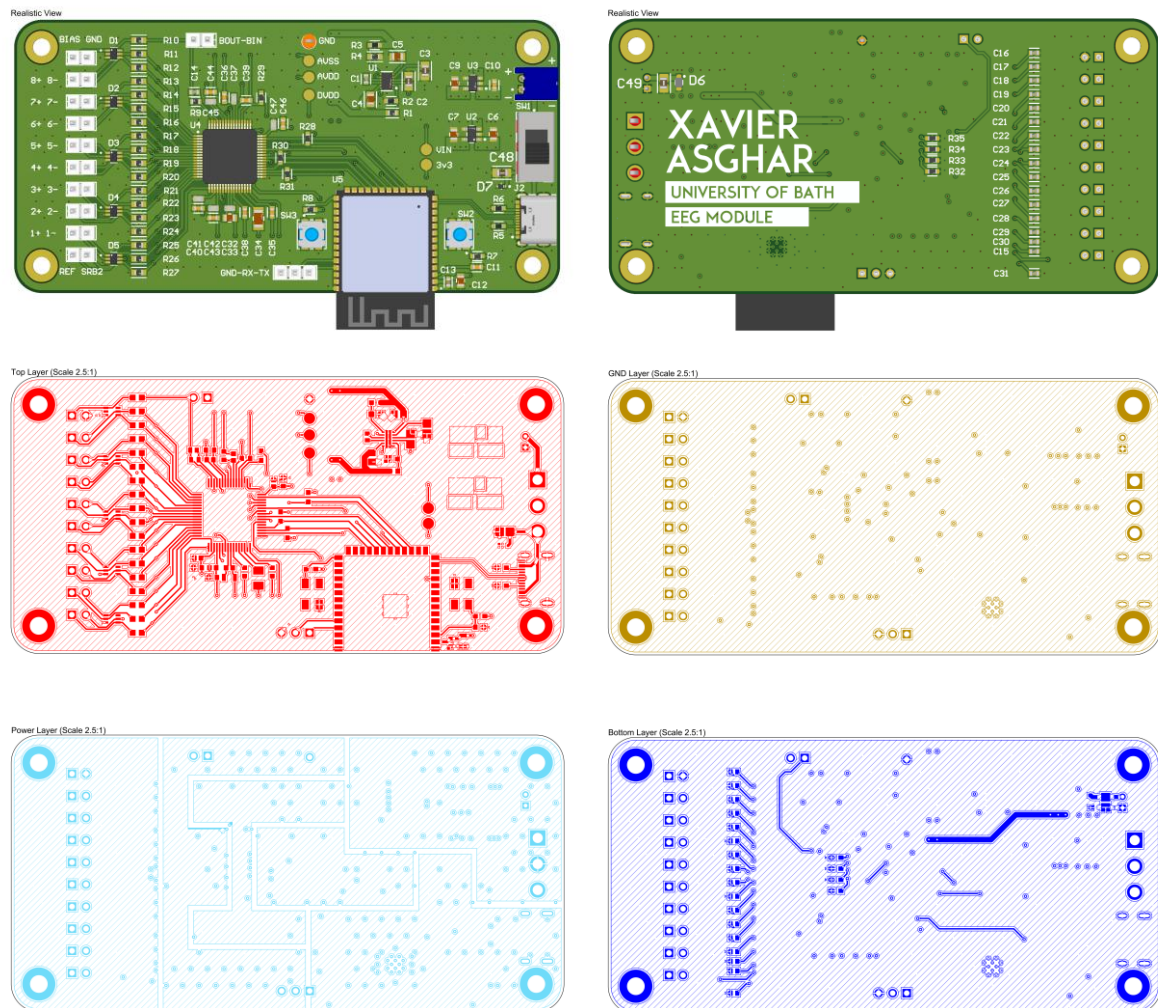


Figure 14: Complete PCB design: top realistic view (upper-left), bottom realistic view (upper-right), top layer (middle-left), ground layer (middle-right), power layer (lower-left), bottom layer (lower-right).

A segmented power plane has been used to distribute the different supply rails across the board and limit interference from digital rails to the analogue section. Care has been taken not to route signals beneath power plane splits, ensuring all traces have continuous return paths. As per the ADS1299 datasheet, void areas on signal layers have been flooded with ground and stitched with vias to the ground plane, and decoupling capacitors have been placed as close as possible to their respective power pins.

Five boards and one stencil were purchased from JLCPCB for a total of £18.95, including customs duties and tax, but excluding shipping. The boards were ordered with an ENIG (Electroless Nickel Immersion Gold) surface finish instead of HASL (Hot Air Solder Levelling) to guarantee a completely flat surface, which is important for fine pitch SMD components. Additionally, unlike HASL, ENIG finishes are resistant to corrosion and oxidation [57]. The unassembled PCB can be seen in Figure 15.1.

The PCB components were ordered from Mouser Electronics, coming to a total component cost of £99.92, which is already an order of magnitude cheaper than the OpenBCI Cyton board. The cost can be lowered to only £68.07 by substituting the 8-channel ADS1299 with its 4-channel equivalent, the ADS1299-4. With the intended use in a primarily educational environment, 4 channels are more than sufficient to detect a range of EEG features, as is detailed in Chapter 5. The full bill of materials (BOM) can be found in Appendix F.

The board was subsequently assembled, with the top layer components being attached using solder paste and a reflow oven, while the bottom mounted and through hole components were soldered by hand. The assembled PCB can be seen in Figure 15.2.

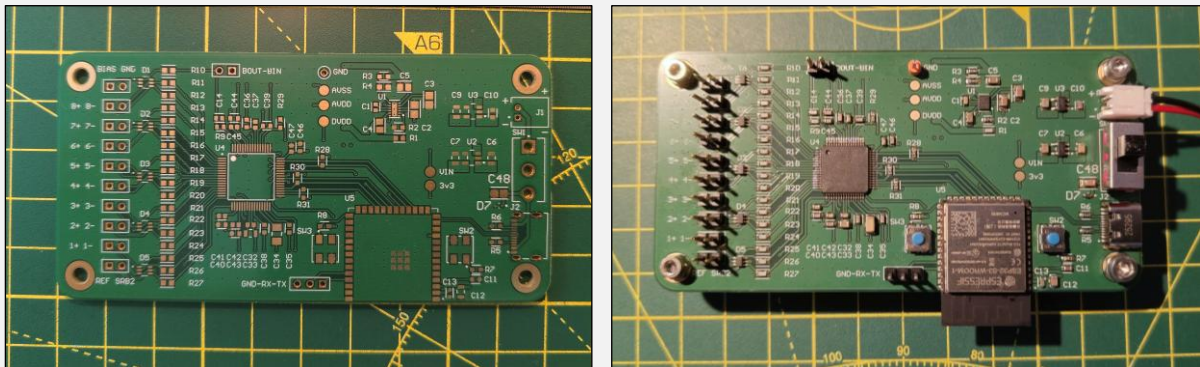


Figure 15: Unassembled PCB (left), and assembled PCB (right)

4.5. Mechanical Design

For the final stage of the design process, a custom enclosure, shown in Figure 16, was designed in Autodesk Inventor to house the electronics. The intention was to elevate the PCB from the work surface, as well as providing a removable lid to protect the device when in transport.

The PCB is bolted to the base through its M3 mounting holes. When open, the top of PCB has been left entirely uncovered, facilitating easy access to the pin headers, buttons and test points, as well as allowing airflow around the ESP32 for better thermal performance. Once again clearance has been given under the ESP32-S3 antenna, as well as around the USB and JST connectors to allow for easy access.

The lid is locked into place using a snap fit, created by four 2.5mm dimples on the base, and corresponding divots on the inside lid. The lid has a lip running around its midpoint, such that when combined with the lip on the bottom of the base, gives the user grip points, allowing for easy removal.

The enclosure was 3D printed on a Formlabs SLA (stereolithography) printer, with white resin V4.1.

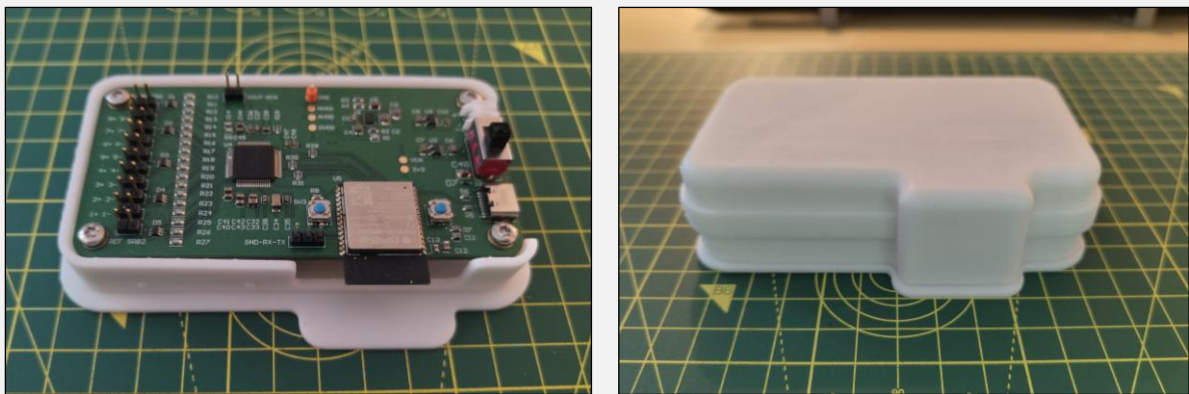


Figure 16: PCB enclosure with lid off (left) & lid on (right)

5. SYSTEM VERIFICATION

As previously mentioned, EEG signals have a magnitude of the order of microvolts, and thus are challenging to measure. Therefore, initial testing focuses on measuring larger biopotentials, in order to prove basic functionality of the system, and bring to light any unexpected issues. Only after this EEG testing is carried out.

5.1. Electromyography (EMG) Testing

Electromyography measures the electrical activity of skeletal muscles, specifically by measuring the superposition of Motor Unit Action Potentials (MUAPs), generated during voluntary contraction. Surface EMG can range between $\pm 5000 \mu\text{V}$, and directly reflects the force output of the involved muscle, as exertion of greater force increases both the recruitment and firing frequency of the MUAPs [58].

For the initial test of the EEG module, a forearm EMG test was carried out, where the active electrode was placed on the muscle belly of the medial aspect of the right forearm, specifically targeting the flexor digitorum superficialis. The reference electrode was placed on the back of the right wrist, so that it is on electrically inactive tissue, while the bias was placed on the back of the right hand [59]. The electrodes were applied using the following procedure [58], [60]:

1. Skin at electrode locations wiped with 70% Isopropyl Alcohol (IPA) wipes, to remove oil and sweat.
2. Skin rubbed with woven gauze swabs, to remove dead skin cells, dirt, and residual IPA.
3. Electrode cups filled with Ten20 conductive paste to form a dome shape.
4. Electrodes affixed to target locations using medical tape
5. Electrode lead wires taped to arm with sufficient slack to prevent mechanical pulling and provide strain relief.

To evoke EMG responses of varying magnitudes, the subject clenched their hand at three distinct velocities: fast, medium, and slow. The raw data was recorded using the LSL Lab Recorder application, and was subsequently processed in python, where a 50 Hz notch filter was applied to eliminate mains frequency interference, and a bandpass filter (20 – 125 Hz) was used to attenuate baseline and high frequency noise. Figure 17.1 shows a labelled example of an EMG recording from the biceps brachii [58], illustrating a noise free baseline, interrupted by EMG spikes of random shape, establishing a morphological example for this test to recreate. The test setup can be seen in Figure 17.2, where the two electrodes shown are reference and bias.

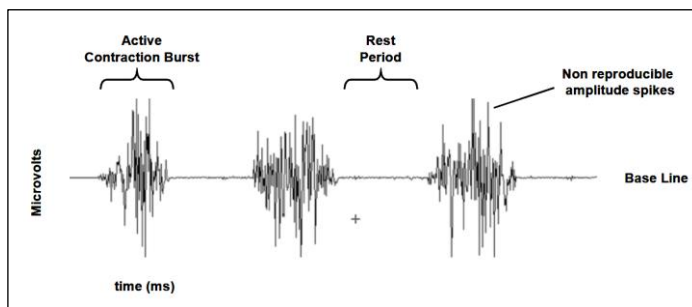


Figure 17.1: Example EMG Reading [58]

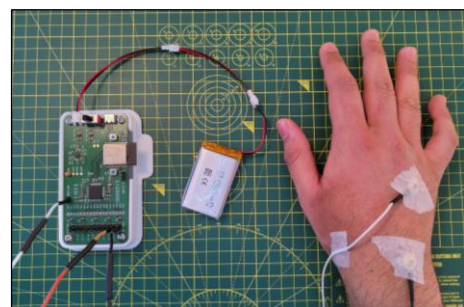


Figure 17.2: EMG test setup (showing ref & bias)

Figure 18 shows the recorded EMG results, and demonstrates that the system successfully captured the physiological relationship between contraction speed and motor unit recruitment. During the fast contraction, a dense cluster of MUAPs can be seen overlaid on top of each other, with an amplitude of $\pm 1500 \mu\text{V}$. As the amount of force decreased, the peak amplitudes proportionally reduced to $\pm 1000 \mu\text{V}$ for the medium speed contraction, and $\pm 400 \mu\text{V}$ for the slow contraction. Crucially, the periods of rest between contractions demonstrate a flat baseline, proving that the system is working as intended.

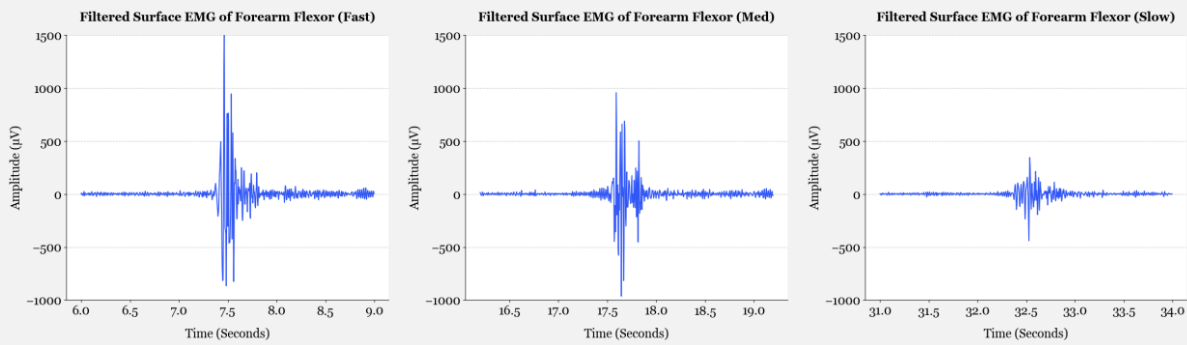


Figure 18: EMG hand clench test results (fast, medium, slow)

5.2. Electrocardiography (ECG) Testing

The initial EMG test was followed up by a supplementary ECG test, in order to validate the system's ability to accurately capture biopotentials with distinct, recurring morphological features, as opposed to random bursts of muscle activity. Additionally, ECG waves are typically slightly smaller than those from EMG, at around $\pm 2000 \mu\text{V}$.

Electrocardiography measures the electrical activity of the heart, and is most commonly used for the assessment of cardiovascular disease. Figure 19 illustrates the basic structure of an ECG wave, where each feature corresponds to specific stages of atrial and ventricular depolarisation and repolarisation [61]. Data acquisition was conducted using the standard Einthoven's Triangle

configuration. The active and reference electrodes were positioned on the right and left wrists, while the bias electrode was placed on the right ankle. Unlike EMG, where the reference is placed at a point of electrical neutrality, the heart can be measured as a single time-varying dipole, and thus a differential signal can be read between the right and left arms [18, p. 231]. The skin preparation procedure outlined in the previous section was also adhered to in this test.

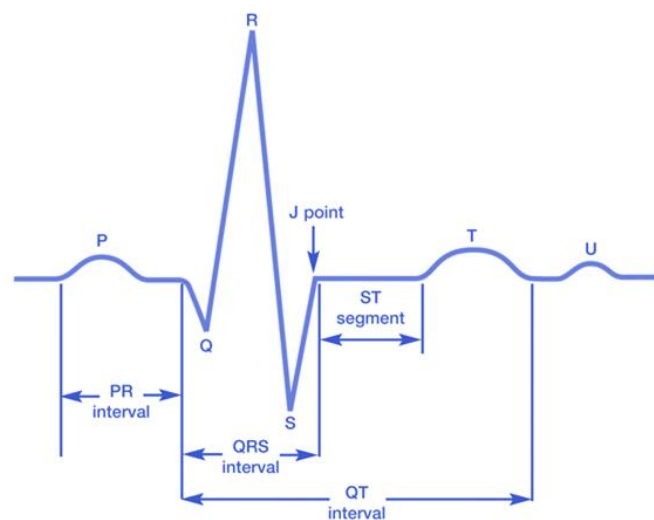


Figure 19: Simplified and labelled ECG wave [61]

Figure 20 presents the filtered ECG waveforms captured during this test. Once again a 50 Hz notch filter has been applied, but this time paired with a bandpass of 0.5 – 40 Hz, in order to preserve the parts of the signal that exist in the lower frequencies [62]. The P wave, QRS complex, and T wave can be clearly seen, occurring approximately every second, corresponding to a healthy resting heart rate of 60 BPM. Overall, the system has recorded a clean ECG wave, once again verifying its performance.

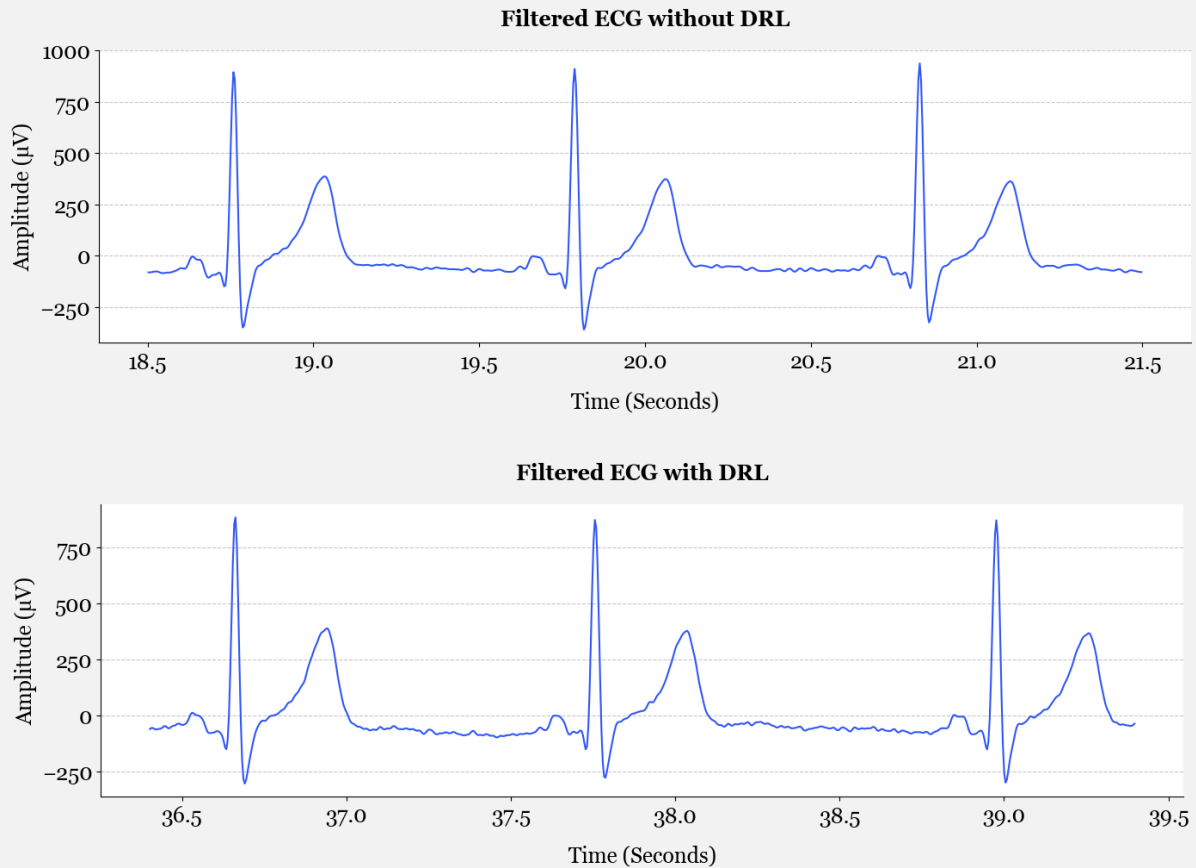


Figure 20: ECG test results, with and without driven right leg enabled

Furthermore, Figure 20 evaluates the impact of the driven right leg circuitry, contrasting recordings with the DRL disabled (top) and enabled (bottom). Surprisingly, activating the DRL slightly degraded the signal quality, introducing marginally more high frequency noise into the waveform. This observation aligns with the findings of Gomez-Clapers et al. (2011) [63], who suggested that in isolated, battery powered amplifiers with high CMRRs, the use of a DRL can increase differential-mode interference, while providing no real benefit, as common mode noise is already at a minimum. This is because the DRL redistributes how power-line currents flow through the body in an unpredictable way. Therefore, subsequent testing should be conducted with the DRL disabled, or should at least compare results with and without the DRL.

5.3. Alpha Wave Test

Alpha waves are 8 to 12 Hz oscillations that typically occur in relaxed states, with minimal mental activity [64]. It was initially believed that the presence of alpha waves reflected rest or idling, however contemporary research demonstrates that they represent the functional inhibition of parts of the brain not needed for specific tasks, and more broadly the allocation of computational resources within the brain [65]. As the most dominant oscillation in the human brain [66], alpha waves provide an ideal physiological benchmark for validation of the EEG module.

To capture these signals, active electrodes were placed at O1, O2, and Oz, in accordance with the 10-20 system. This configuration targets the occipital lobe; the primary centre for visual processing. The reference and bias electrodes were placed on the electrically quiet left and right earlobe. Once again electrodes were applied using the process outlined in Section 5.1, this time held in place with a section of tubular bandage. The experimental setup can be shown in Figure 21.



Figure 21: Alpha wave test setup

The experiment leveraged the alpha blocking effect, whereby alpha wave amplitude is significantly attenuated when the eyes are opened [67]. The test consisted of a 20 second baseline period, where the subject fixated on a blank wall, followed by an auditory cue to signal the closing of the eyes for another 20 seconds. To ensure precise timing of the data collection, the test script pushed synchronisation markers to a dedicated LSL stream, marking the start and end of each phase of the experiment.

Following data acquisition, a 50 Hz notch filter and a 1 to 50 Hz bandpass filter were applied. The power spectral density was then obtained using Welch's method, which divides the data into windowed segments, takes the discrete Fourier transform of each section, and then averages the resulting spectra [1, p. 96] [68]. This averaging process has the benefit of cancelling random noise, whilst preserving continuous signals. The resulting spectral comparison is presented in Figure 22.

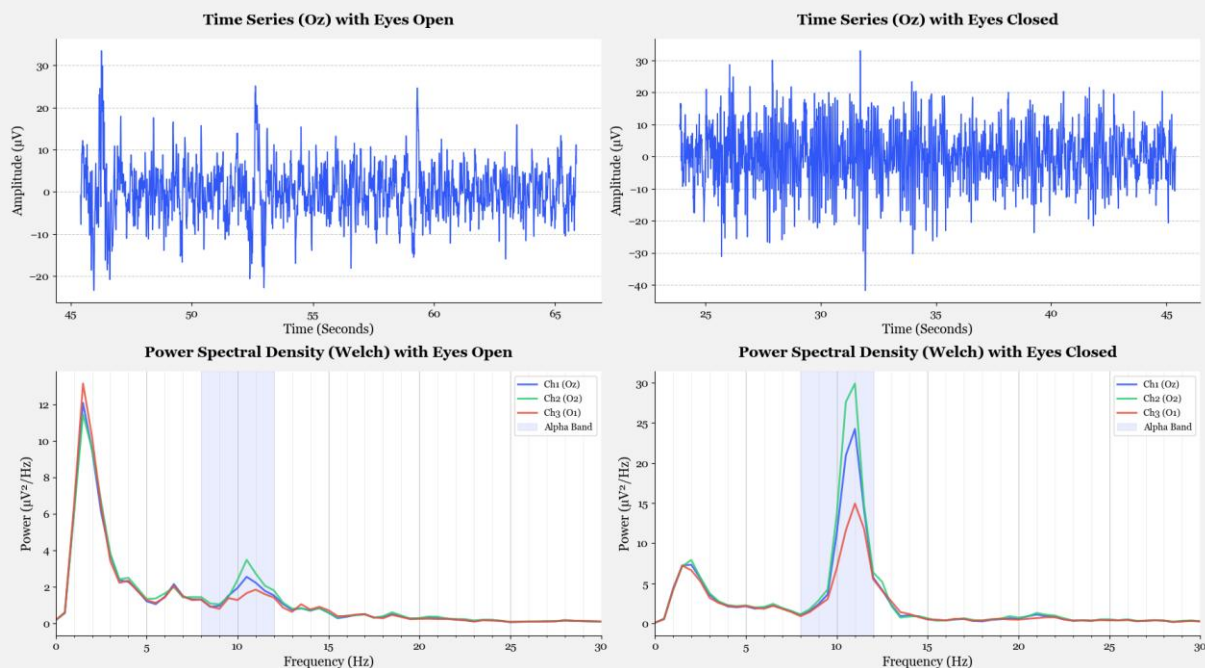


Figure 22: Time and frequency domain comparison of occipital alpha waves recorded with eyes open vs closed

As anticipated, there is a significant increase in alpha wave power during the eyes closed section of the test, with the greatest magnitude seen at the O2 position. This aligns with findings from J. Samaha & B. Postle (2015) [69], who similarly observed that alpha power peaked at the O2 position during their investigation into the temporal resolution of visual perception. The successful acquisition of alpha waves represents a critical milestone in validating the effectiveness of the proposed EEG module.

5.4. SSVEP Test

The next verification phase involved the detection of steady state visually evoked potentials (SSVEPs), which are evoked responses induced by flickering visual stimuli [70]. An SSVEP response appears as an increase in the amplitude at the fundamental frequency of the presented stimuli, often accompanied by peaks at its harmonics. As well as being used in neuroscience, to diagnose visual pathway and brain mapping impairments, SSVEPs are often used as a basis for BCI applications [24]. For example in BCI spellers, users navigate interfaces by focusing on specific flickering targets [71]. It thus follows that the reliable detection of SSVEPs is important for an EEG module designed to serve as a BCI teaching tool.

This test utilised the same occipital electrode configuration as the previous alpha wave test. Visual stimuli were generated on a 120 Hz monitor, using the Pygame library to alternate the screen between black and white in synchronisation with the monitor's refresh rate. Target frequencies were achieved by changing the screen state after a specific number of frames, provided the target frequency was an even integer division of the 120 Hz refresh rate. It is for this reason that SSVEP responses were tested at 6, 7.5, 10, 12, 15, 20, 24 and 30 Hz. For each test the subject would stare at a marker on a black screen for 5 seconds, before being presented with the flickering stimuli for 10 seconds. Once again, LSL markers were used to precisely timestamp the beginning and end of each test stage.

Figure 23 (next page) shows the Fast Fourier Transform (FFT) of the filtered EEG data captured from each test. To prioritise frequency resolution over noise reduction, a standard FFT was used in place of Welch's method. For each stimuli frequency we can observe the resulting peak in the EEG data, and at lower frequencies, including 6, 7.5 and 10 Hz, harmonic peaks are also present. The most powerful responses were elicited at 12 to 15 Hz, which is expected as these frequencies align closely with the alpha wave frequency range, perhaps demonstrating a resonance phenomenon [70]. At the extremes of the tested spectrum (6 & 30 Hz), the response has decreased in magnitude.

While the SSVEP detection was largely successful, specific anomalies can be observed in the 20 and 30 Hz tests, where despite the presence of a target frequency peak, there are other dominant frequencies. Because these artifacts were repeatedly observed at these frequencies with this specific visual setup, and did not occur with alternative testing configurations, they are likely due to limitations of the software-based test setup leading to errors in the presented visual stimuli. For instance, dropping frames would alter the effective frequency displayed to the user. It is for this reason that future iterations of this test would be better conducted by flickering an LED driven by a microcontroller to guarantee accurate stimulus frequencies.

Overall, the module's ability to isolate SSVEP frequencies validates the systems performance, and confirms its readiness for use in BCI-applications.

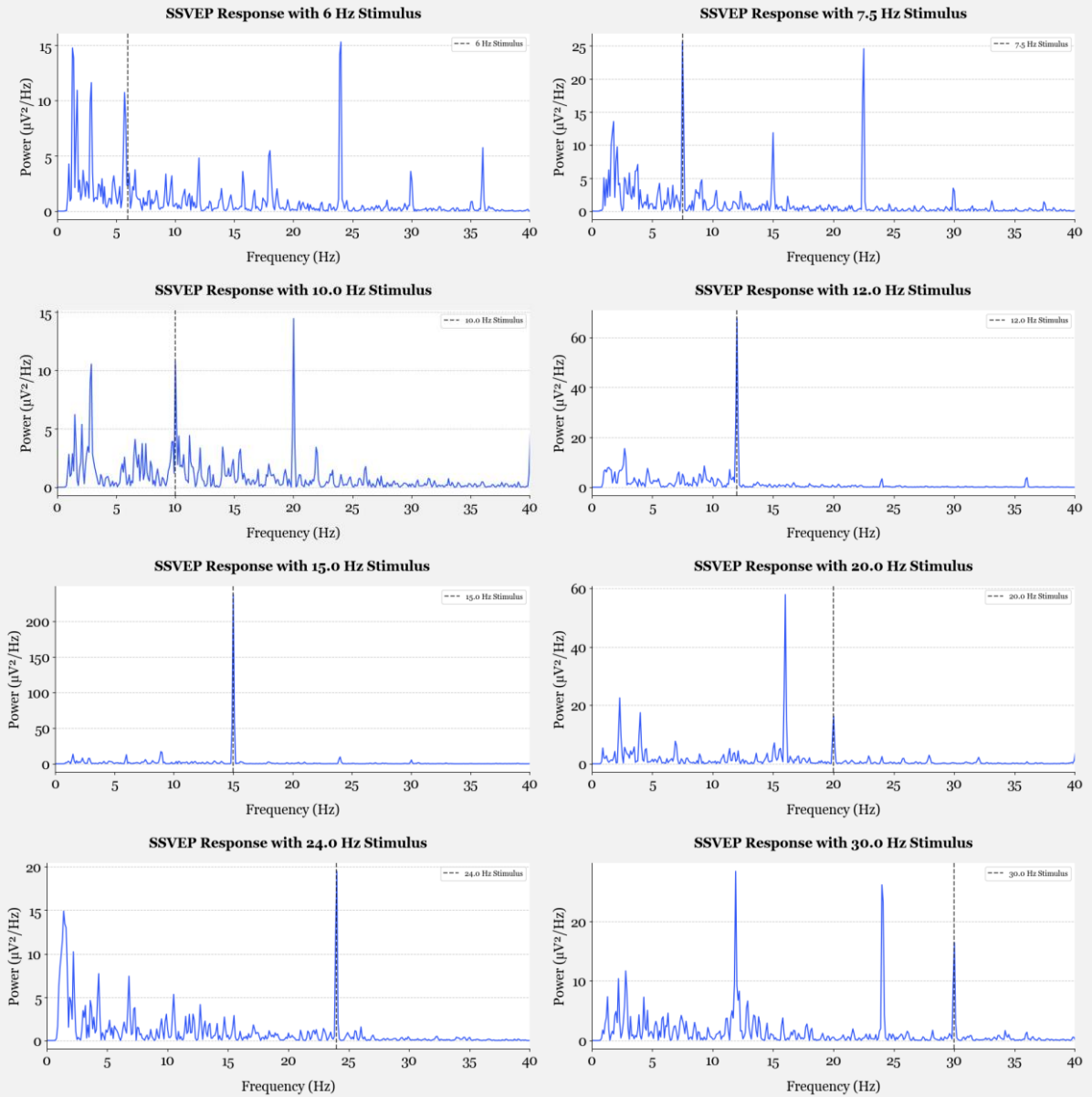


Figure 23: SSVEP response spectra with varying stimuli

5.5. P300 Test

The P300 is a positive deflection occurring approximately 300 milliseconds after a subject detects an informative, task-relevant stimulus [38]. It is most commonly elicited through the use of an auditory oddball paradigm, where the subject is required to count mentally, and to distinguish a more infrequent target tone from a nontarget stimulus [72]. The P300 is produced by a distributed network of brain processes associated with attention and memory operations, although there is not yet a clear understanding of exactly how and why the brain produces the P300 [25].

In contrast to alpha waves and SSVEPs, the P300 is a specific time domain waveform, and thus accurate P300 detection presents a greater challenge for the EEG module. In addition, P300 ERPs have long been used as the basis for BCI systems, for instance Farwell and

Donchin's P300 speller (1988) [73], which relies on a P300 being produced when a letter the subject is looking at is flashed.

The subject was played a recording consisting of 1000 Hz standard tones, and 2000 Hz target tones. In total 100 tones were played at intervals between 1.4 and 1.8 seconds, of which 20 to 30 were the target tones. The resulting responses to the standard tones and the target tones were notch filtered at 50 Hz, bandpassed between 0.5 & 20 Hz, and subsequently averaged to remove random noise. These results are shown in Figure 24, where the standard error of the mean has also been plotted to show the variance of the results.

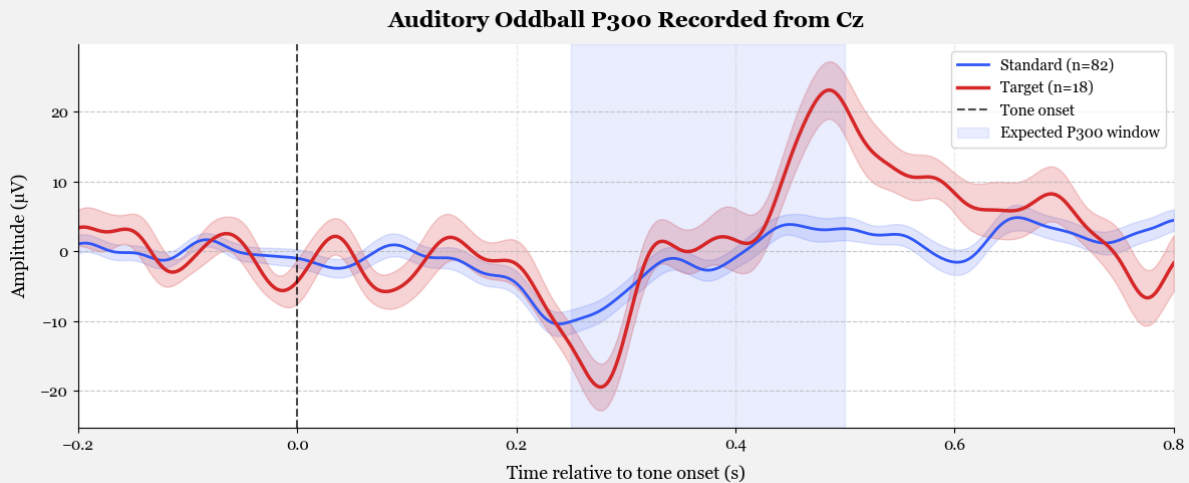


Figure 24: ERP response evoked from target tones ($n=18$) compared to standard tones ($n=82$)

Figure 24 shows a clear positive deflection within the expected P300 window, peaking at 23 μV at 485 ms, during which the standard waveform is at the baseline. The P300 waveform has a marginally larger standard error of the mean, however this is expected as P300s show high variability due to changes of cognitive state within an experimental session [74].

For the sake of comparison, Figure 25 shows an example P300 waveform, from a study comparing different P300 paradigms with different aged subjects [72]. These results were obtained with an almost identical experimental setup, but represent the average from twenty 100-tone trials, taken from children aged 12-14 years old.

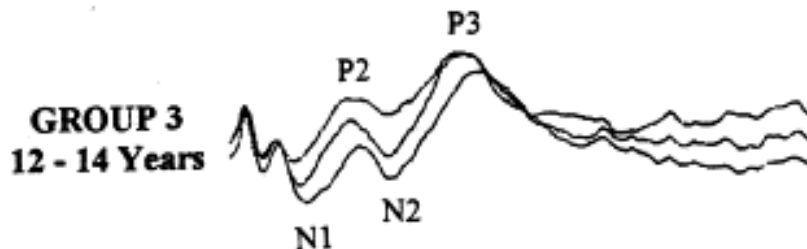


Figure 25: Example P300 recording averaged from 20 individuals [72]

In both Figure 24 waveforms, an N2 peak can be seen in the region of 200 to 300 ms. These appear across both the standard, and target waves due to the fact they are sensory, rather than cognitive evoked potentials. The recorded P300 has very similar morphology to the example shown in Figure 25, and subtle differences can be attributed to the greater number

of trials the example data is averaged across, as well as the fact that P300 can vary depending on the subject and the conditions.

In addition, other time domain ERP tests were conducted, including tests for flash VEPs (visually evoked potentials), pattern reversal VEPs, as well as half-field pattern reversal VEPs, the results of which can be found in Appendix G. Taken together with the results of the P300 test, they confirm the system's ability to measure specific time series EEG waveforms, and provide results aligned with those expected from the literature. This test completes the chain of verification, and proves the module is capable of being used as a brain computer interface, particularly since SSVEPs and P300s are commonly used as the basis for BCI systems.

Beyond standard BCI applications, the P300 can also be used in a forensic capacity, as a method of identifying concealed information. The results of carrying out such a test are detailed in Appendix H, which is intended to serve primarily as a demonstration of this application, rather than another stage of verification.

6. DISCUSSIONS & CONCLUSIONS

6.1. Evaluation Against Objectives

The overall aim of this project was to design, build and validate a low-cost, open-source EEG module capable of safely measuring known EEG features for educational and research applications. Evaluating the system against the four core objectives outlined in section 2.2 demonstrates that the project has been largely successful, and can genuinely help bridge the gap between expensive commercial EEG headsets, and the poorly documented academic prototypes.

Objective 1: Hardware Design and Assembly

Fully Met

A custom EEG PCB module was designed, assembled, and tested, integrating an ADS1299 analogue front end with an ESP32-S3 to accurately measure EEG signals from input electrodes, and transmit them to an external computer. Patient safety is maintained through the use of battery power and wireless communication. Most importantly, the low-cost constraint was satisfied, with a total component cost of £68.07 for 4 EEG channels, or £99.92 for 8 channels, representing an order of magnitude reduction in cost compared to commercial equivalents, like the OpenBCI Cyton.

Objective 2: Software and Firmware Integration

Fully Met

A robust data transmission pipeline has been implemented, capable of recording EEG data at 500 SPS, without any packet loss, and sending the data to the Lab Streaming Layer. This allows the user to use free LSL applications including BrainVision, Lab Recorder, and OpenViBE, to view and filter the data in real time, or to record the data alongside time stamped external stimuli.

Objective 3: System Validation

Fully Met

The system's EEG acquisition capabilities were evaluated across a range of human biopotentials, starting with noise free recordings of EMG and ECG waves, and ending with the successful detection of EEG features commonly used in BCI systems, including alpha waves, SSVEPs, and P300s. In particular the acquisition of evoked potentials of similar scale and morphology to those in the literature reinforces the system's readiness to be used as a brain computer interface. Additionally an input referred noise of $1.10 \mu V_{pp}$ was measured with the channel inputs shorted, demonstrating a comparable performance to the ADS1299 datasheet.

Objective 4: Open-Source Documentation

Partially Met

Contained in the appendices of this report are the complete firmware and software source code, as well as the Altium schematic, and BoM for the module. However, a complete, open-source, project repository, that would also contain Gerber files for the board layout and STL files for the enclosure, has yet to be created, due to the fact there are minor improvements that ideally would be made before the designs are released to the public (see section 6.4). Moreover, due to timing constraints, user guides and documentation have not yet been written.

6.2. Suitability for Target Application

This prototype was developed to address a gap within the BCI landscape: the lack of a truly low-cost, fully open source, and easily reproducible EEG module. At present, should you wish to purchase an EEG machine, you are forced to spend hundreds of pounds on what are often fully integrated, inflexible systems. By utilising the industry standard ADS1299 analogue front end, the system can achieve the signal fidelity of a premium headset, but at a fraction of the hardware cost.

One area where this high cost is a particularly large barrier is in educational environments, where the extreme cost of EEG hardware puts it out of reach of most University modules and research projects. Table 6 compares the price of the proposed system to the OpenBCI Cyton board, when purchasing EEG modules for a class of 20 students. The total price is diminished from a completely unaffordable £18,660, to £1,402.65. While still making for an expensive purchase, this is significantly closer to being a realistic option for a university, although it would also require time from a lab technician for board assembly.

Table 6: 20 module price comparison vs OpenBCI Cyton Board

	Proposed EEG Module	OpenBCI Cyton
Individual Cost	£68.07	£933
Total Component Cost	£1,361.40	£18,660
Fabrication Cost	£41.25 (JLC PCB)	£0.00
Total Cost	£1,402.65	£18,660

Furthermore, the component cost of £68.07 is well within the budget for a typical university final year project, or group business and design project, opening up possibilities for students to implement brain computer interfaces into their engineering projects, or to gather EEG data as part of a study. This hands-on experience would be invaluable for teaching students about brain computer interfaces, as well as broader topics such as signal processing, feature extraction and machine learning, all of which are fundamental to BCI technology.

The module itself is well suited for use in these kinds of educational environments, with standard 2.54 mm pin header electrode connections allowing for the use of any type of electrode, which can subsequently be placed in any configuration. This is contrary to devices like the Muse 2, or Emotiv Epoc X, which lock you into using a single type of electrode, in fixed positions on the head. On the software side, the integration with the Lab Streaming Layer makes it easy to synchronise tests involving external stimuli, as well as giving users the flexibility to view and process the data how they see fit. In particular, the OpenViBE toolbox is particularly well suited, allowing students to process the data in real time with an intuitive drag and drop interface.

6.3. Limitations and Sources of Uncertainty

While the prototype successfully met its core objectives, several limitations and sources of uncertainty were identified during the testing process, highlighting areas for refinement.

The first major source of uncertainty is the lack of quantitative metrics by which to assess the EEG module. While extensive EEG testing was carried out, proving that the module could discern several well-known EEG features, the testing process would greatly benefit through comparisons with commercial and clinical EEG machines, as well as multi-subject trials. Running the same tests on other proven EEG hardware would reduce any uncertainty in the test results, for example proving that the anomalous SSVEP results acquired at 20 and 30 Hz resulted from an inadequate test setup, rather than a hardware flaw. Moreover, having a true benchmark for each of the conducted tests would allow for a better assessment of exactly how accurate each of the results are, as opposed to the current, somewhat binary system of just checking if a feature can be detected at all. This holds true particularly for tests like the P300 test, where the results typically show a large person to person variance.

Another source of uncertainty are potential time offsets in the recorded data. Results from the flash VEP test and pattern reversal VEP test, located in Appendix G, as well as the P300 test, show the recorded wave consistently occurring around 25-100 ms later than set out by the literature. A possible reason for this delay is that BLE is not a particularly time sensitive communication protocol, and could be introducing delay or jitter to the recorded data. A possible solution to this would be to timestamp the data as soon as it is read by the ESP32, and force the LSL to use this timestamp, rather than letting it create one from the PC's clock when being pushed. Another limitation caused by BLE is a 500 SPS data limit, whereby if the ADS1299 sampling rate is increased to 1000 SPS, packets begin to be dropped. While this is not a major drawback in an educational environment, as 500 SPS is more than adequate to record standard EEG features, it prevents the user, should they wish, to record higher frequency data.

Finally, testing of the EEG module has been conducted solely with gold cup electrodes, which while extremely cheap and reliable, are not particularly well suited to an educational environment. This is because the required skin preparation and application of conductive paste can be a lengthy and slightly messy process, which would leave the user with residual paste in their hair after testing. Moreover, purchasing conductive paste and other skin preparation equipment incurs additional costs, in particular the industry standard Ten20 paste costing a minimum of £20 per 228g tub. The use of dry electrodes would entirely negate this issue, allowing for fast and easy application, at the cost of signal fidelity. For this reason, it would be worth sourcing a set of dry electrodes, and testing the capabilities of the system with these in comparison to gold cup electrodes.

6.4. Future Work

To evolve the initial prototype into a system that is ready for more widespread deployment, several avenues for future development have been identified.

Firstly, as is often the case with PCB design, there are several small improvements that should be made to the board before the design is finalised:

Power Path Management

High Priority

The most pressing change is adding power path management, such that the USB-C power rail does not power the ADS1299, only allowing the user to record data while the device is battery powered, and forcing electrical isolation. For initial testing and debugging it was helpful to be able to power the entire system over USB, however relying on the user to manually switch to battery power is not a sensible long term design choice.

Design for Manufacture

High Priority

In the current design, a limited number of components have been placed on the bottom layer in order to reduce PCB size. However, this unnecessarily lengthens the assembly process, as these components then need to be hand soldered. For the same reason, where possible through hole components should be replaced by surface mount counterparts. Both of these changes also reduce the cost of purchasing the board assembled.

On-Board Charging

Medium Priority

Adding battery charging functionality to the board, for instance with a low-cost constant current/constant voltage charging IC like the MCP73871, would remove the need for external LiPo charging modules. This is a straightforward change, as the board already has a USB-C port that could be used for charging. Additionally, the ICs power good pin could be used to accomplish the power path management change, by routing it to the enable pin of the ADS1299 LDOs.

Daisy Chain Functionality

Low Priority

The ADS1299 can be set up such that daisy chaining multiple modules together is possible. While not strictly necessary for use in education, exposing the correct daisy chaining pins, and allowing the user to create a system with more than 8-channels, would open up a small amount of additional applications.

Aside from hardware changes, there are a couple of software and firmware changes that could be implemented. Firstly, as discussed in the above section, it would be worth changing the firmware to timestamp the ADS1299 samples as soon as they are read, to remove the potential for timing offsets. Secondly, it would be beneficial to create a simple GUI for the python host script, removing the need for the user to interact with the windows terminal to see the lead off bits and other status information. This GUI could also be expanded to allow the user to change ADS1299 setup registers, rather than having to change them manually in the ESP32-S3 source code, facilitating straightforward adjustment of settings including PGA gain, or driven right leg derivation channels. This would involve the implementation of bidirectional BLE communication.

Finally, the most important piece of future work is the compilation of all of the source code, schematics, Gerber files and STL files, into a comprehensive repository, allowing for the project to be easily reproduced, and thus made accessible to other educational and research institutions. This must be paired with thorough documentation, explaining how the system was created, how it should be used, and notifying the reader of any identified limitations. The repository could also include the test scripts used to detect EEG features identified in the report, to act as demonstrations of the system, and aiding the user in their own development.

6.5. Conclusion

In conclusion, this project has delivered a low-cost, wireless EEG module that addresses a clear gap in the current BCI landscape, bringing down the cost of EEG technology to a level suitable for educational deployment. With a minimum component cost of £68, the proposed system is significantly cheaper than any comparable commercial offerings, and has been validated across a range of features that underpin BCI systems, including alpha waves, SSVEPs and P300 responses. The combination of the ADS1299 analogue front end, the ESP32-S3 BLE capable microcontroller, and the LSL integration, produces a flexible, electrode agnostic system that would be well positioned for adoption into student-led projects and undergraduate teaching. With the refinements outlined above, particularly the PCB design changes, the ESP32-S3 timestamping, and the open-source repository, the module is ready to make hands-on EEG learning substantially more accessible.

7. PROJECT REVIEW

7.1. Project Scheduling and Scoping

Generally speaking, a waterfall development process has been followed during the project. This process was chosen due to the fact the project has clearly defined requirements, and is highly structured by nature [75]. For instance, once the finalised PCB design is sent for manufacture, it becomes difficult to make large changes to previous design stages. The overall project structure, and its adherence to the 5 stages of the waterfall model, can be seen in Figure 26.

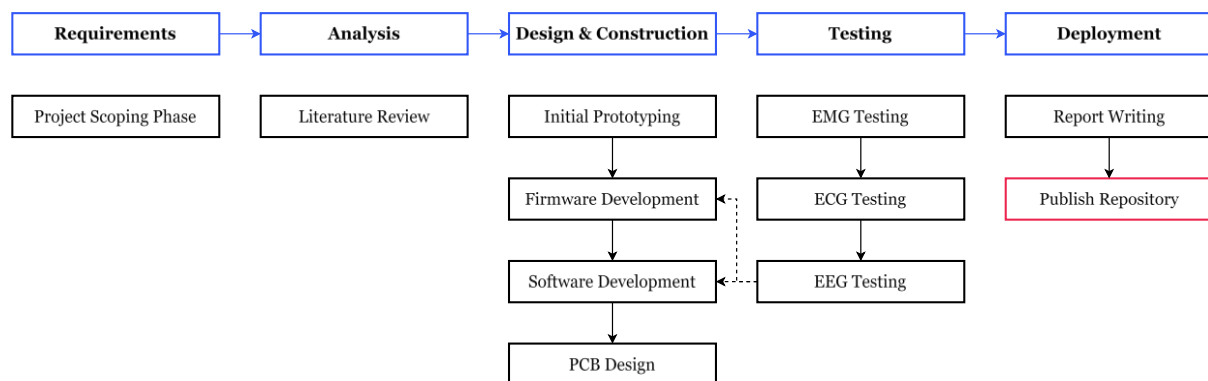


Figure 26: Overall project structure, divided into waterfall stages

During the project scoping phase, a Gantt chart was created, however the project fairly quickly deviated from the original development plan, as the need for a custom PCB, rather than a development board prototype became apparent. The introduction of a PCB design stage placed stress on the project timeline, as naturally the board took time to manufacture and assemble. This removed any margin for error within the project, as in the case where there was a fault in the PCB design, there would be insufficient time and budget to get it manufactured again. Fortunately, the PCB design and assembly was one of the main successes of the project, with the design functioning exactly as intended. This is what facilitated the acquisition of microvolt EEG signals, ultimately underpinning the project's overall success. However, it is still worth noting that with proper planning, this pressure could have been somewhat alleviated.

7.2. Supply Chain and Logistical Challenges

The most significant obstacles during the project arose from difficulties sourcing critical components. For instance, the literature review had established the ADS1299 as the standard for low-cost EEG boards, however its evaluation module was out of stock for multiple weeks. Due to the sequential structure of the waterfall process, this delay to the start of the development process propagated through to each subsequent stage. Furthermore, sourcing EEG specific components, like Ten20 conductive paste, or gold cup electrodes, from UK suppliers proved difficult, with both of these taking weeks to be delivered, and without which the project would not have been a success. The effects of these delays were then compounded when an administrative error resulted in an unforeseen weeklong delay of the PCB components.

Once again, this has highlighted the importance of proper scheduling. Incorporating buffer periods into the project schedule to allow for procurement delays, as well as ordering critical components earlier in the development cycle, would have resulted in more time for the latter stages of the project, as well as a greater margin for error were something to go wrong. Ultimately, despite the project's overall success, better time management and planning may have resulted in Objective 4: Open-Source Documentation, being fully complete, as there would have been sufficient time to create documentation.

7.3. Budgetary Constraints

A final challenge throughout the project lifecycle was the management of the development budget. The initial allocated budget of £250 quickly proved insufficient for the required hardware prototyping stage, with the ADS1299 evaluation module alone costing £194.23. A budget extension of £60 was secured from the university in order to afford the components to populate the PCB, however, to complete the system integration and testing, further expenditure was necessary. From this point the Ten20 conductive paste, as well as gold cup electrodes were purchased by my project supervisor, the PCB manufacture was financed through personal funds, while the enclosure was printed for free by the university. The full project expenditure can be found in Appendix A.

This experience highlighted the fact that the development of a low-cost product can far exceed its eventual unit cost. While the final BoM for a 4-channel module comes to just £68, the ADS1299 evaluation module consumed over three times the cost, before a single component had been ordered. A more thorough budgetary forecast at the start of the project would have removed the need for personal expenditure and unplanned funding requests.

8. ACKNOWLEDGEMENTS

First, I would like to thank my supervisor, Dr Nic Zhang, for providing a project that is both interesting and engaging from a technical point of view, but also due to its broader scientific application. Learning about electroencephalography, and working up to recording my own EEG data, has been a genuinely enjoyable and rewarding experience, and his guidance and support throughout the project are greatly appreciated.

I would also like to thank GE Aerospace Bristol, my placement company, for allowing me to return to use their electronics facilities to assemble my PCB. In particular, I am grateful for Jamie Adams and Lorenzo Colo's assistance and advice during the assembly process, which was invaluable for the completion of my project.

Finally, I would like to thank my housemate Tom Kirsch, for volunteering as a test subject, as well as Rebecca Loundes and Roberta Griffiths for assisting me with electrode application throughout the testing process.

Generative AI Statement

I acknowledge the use of Google's Gemini 3.1 Pro model during the development of the project and writing of this report. Specifically, the tool was used to assist with the following purposes:

- Identifying relevant literature and summarising background concepts.
- Proofreading and refining grammatical clarity of the report.
- Assisting with troubleshooting and debugging C++ and Python code.

All original ideas and decisions presented within this document are entirely my own, and all factual information has been properly referenced.

9. REFERENCES

- [1] L. Hu and Z. Zhang, *EEG Signal Processing and Feature Extraction*, 1st ed. 2019. Singapore: Springer Singapore, 2019. doi: 10.1007/978-981-13-9113-2.
- [2] Neuroelectrics, 'The Evolution of EEG: A Journey Through Time', Neuroelectrics. Accessed: Apr. 13, 2026. [Online]. Available: <https://www.neuroelectrics.com/blog/the-evolution-of-eeg-a-journey-through-time>
- [3] M. Soufineyestani, D. Dowling, and A. Khan, 'Electroencephalography (EEG) Technology Applications and Available Devices', *Appl. Sci.*, vol. 10, no. 21, p. 7453, Jan. 2020, doi: 10.3390/app10217453.
- [4] J. Sabio, N. S. Williams, G. M. McArthur, and N. A. Badcock, 'A scoping review on the use of consumer-grade EEG devices for research', *PLOS ONE*, vol. 19, no. 3, p. e0291186, Mar. 2024, doi: 10.1371/journal.pone.0291186.
- [5] M. Duvinage, T. Castermans, M. Petieau, T. Hoellinger, G. Cheron, and T. Dutoit, 'Performance of the Emotiv Epoc headset for P300-based applications', *Biomed. Eng. Online*, vol. 12, p. 56, Jun. 2013, doi: 10.1186/1475-925X-12-56.
- [6] 'EmotivPRO – EMOTIV'. Accessed: Feb. 19, 2026. [Online]. Available: <https://www.emotiv.com/emotivpro>
- [7] 'The Complete Ultracortex', OpenBCI Shop. Accessed: Feb. 19, 2026. [Online]. Available: <https://shop.openbci.com/products/the-complete-headset-eeeg>
- [8] 'Emotiv EPOC X', Emotiv. Accessed: Feb. 19, 2026. [Online]. Available: <https://shop.emotiv.com/epoc-x/>
- [9] A. Jain, I. Kim, and B. J. Gluckman, 'Low Cost Electroencephalographic Acquisition Amplifier to serve as Teaching and Research Tool', *Conf. Proc. Annu. Int. Conf. IEEE Eng. Med. Biol. Soc. IEEE Eng. Med. Biol. Soc. Conf.*, vol. 2011, pp. 1888–1891, 2011, doi: 10.1109/IEMBS.2011.6090535.
- [10] J. Teversham *et al.*, 'Development of an Ultra Low-Cost SSVEP-based BCI Device for Real-Time On-Device Decoding', in *2022 44th Annual International Conference of the IEEE Engineering in Medicine & Biology Society (EMBC)*, Jul. 2022, pp. 208–213. doi: 10.1109/EMBC48229.2022.9871064.
- [11] 'Muse 2 | Muse™ EEG-Powered Meditation & Sleep Headband'. Accessed: Apr. 19, 2026. [Online]. Available: <https://eu.choosemuse.com/products/muse-2>
- [12] E. K. S. Louis *et al.*, 'Introduction', in *Electroencephalography (EEG): An Introductory Text and Atlas of Normal and Abnormal Findings in Adults, Children, and Infants [Internet]*, American Epilepsy Society, 2016. Accessed: Apr. 16, 2026. [Online]. Available: <https://www.ncbi.nlm.nih.gov/books/NBK390346/>
- [13] 'Cerebral Cortex: What It Is, Function & Location', Cleveland Clinic. Accessed: Apr. 16, 2026. [Online]. Available: <https://my.clevelandclinic.org/health/articles/23073-cerebral-cortex>
- [14] 'ADS1299-x Low-Noise, 4-, 6-, 8-Channel, 24-Bit, Analog-to-Digital Converter for EEG and Biopotential Measurements datasheet (Rev. C)'. Texas Instruments, 2017. Accessed: Apr. 16, 2026. [Online]. Available: <https://www.ti.com/lit/ds/symlink/ads1299.pdf?ts=1776326536969>
- [15] 'EEG Montages and Technical Components'. Accessed: Apr. 17, 2026. [Online]. Available: <https://www.learningeeg.com/montages-and-technical-components>
- [16] G. Li, S. Wang, and Y. Y. Duan, 'Towards gel-free electrodes: A systematic study of electrode-skin impedance', *Sens. Actuators B Chem.*, vol. 241, pp. 1244–1255, Mar. 2017, doi: 10.1016/j.snb.2016.10.005.
- [17] 'TMSi — an Artinis company — What Is the 10-20 System for EEG?', TMSi — an Artinis company. Accessed: Apr. 17, 2026. [Online]. Available: <https://www.tmsi.artinis.com/blog/the-10-20-system-for-eeeg>
- [18] J. D. Bronzino, *The Biomedical Engineering Handbook*, 2nd edn, vol. 1. CRC Press LLC, 2000.

- [19] D. Berry, F. Duignan, and R. Hayes, 'An Investigation of the use of a High Resolution ADC as a "Digital Biopotential Amplifier"', in *4th European Conference of the International Federation for Medical and Biological Engineering*, J. Vander Sloten, P. Verdonck, M. Nyssen, and J. Haueisen, Eds, Berlin, Heidelberg: Springer, 2009, pp. 142–147. doi: 10.1007/978-3-540-89208-3_36.
- [20] C. Kothe *et al.*, 'The lab streaming layer for synchronized multimodal recording', *Imaging Neurosci.*, vol. 3, p. IMAG.a.136, Sep. 2025, doi: 10.1162/IMAG.a.136.
- [21] 'Lab Streaming Layer (LSL) | OpenBCI Documentation'. Accessed: Apr. 20, 2026. [Online]. Available: <https://openbci.github.io/Software/CompatibleThirdPartySoftware/LSL/>
- [22] N. M. Jadeja, Ed., 'Frequencies and Rhythms', in *How to Read an EEG*, Cambridge: Cambridge University Press, 2021, pp. 32–39. doi: 10.1017/9781108918923.008.
- [23] A. M. Norcia, L. G. Appelbaum, J. M. Ales, B. R. Cottareau, and B. Rossion, 'The steady-state visual evoked potential in vision research: A review', *J. Vis.*, vol. 15, no. 6, p. 4, May 2015, doi: 10.1167/15.6.4.
- [24] J. D. Chailloux Peguero, L. G. Hernández-Rojas, O. Mendoza-Montoya, R. Caraza, and J. M. Antelis, 'SSVEP detection assessment by combining visual stimuli paradigms and no-training detection methods', *Front. Neurosci.*, vol. 17, May 2023, doi: 10.3389/fnins.2023.1142892.
- [25] J. Polich, 'Updating P300: An integrative theory of P3a and P3b', *Clin. Neurophysiol.*, vol. 118, no. 10, pp. 2128–2148, Oct. 2007, doi: 10.1016/j.clinph.2007.04.019.
- [26] M. P. Barham, G. M. Clark, M. J. Hayden, P. G. Enticott, R. Conduit, and J. A. G. Lum, 'Acquiring research-grade ERPs on a shoestring budget: A comparison of a modified Emotiv and commercial SynAmps EEG system', *Psychophysiology*, vol. 54, no. 9, pp. 1393–1404, Sep. 2017, doi: 10.1111/psyp.12888.
- [27] N. S. Williams, G. M. McArthur, B. de Wit, G. Ibrahim, and N. A. Badcock, 'A validation of Emotiv EPOC Flex saline for EEG and ERP research', *PeerJ*, vol. 8, p. e9713, 2020, doi: 10.7717/peerj.9713.
- [28] 'Emotiv Flex 2 Saline', Emotiv. Accessed: Apr. 19, 2026. [Online]. Available: <https://shop.emotiv.com/flex-2-saline/>
- [29] M. T. Knierim, M. G. Bleichner, and P. Reali, 'A Systematic Comparison of High-End and Low-Cost EEG Amplifiers for Concealed, Around-the-Ear EEG Recordings', *Sensors*, vol. 23, no. 9, p. 4559, May 2023, doi: 10.3390/s23094559.
- [30] 'OpenBCI Featured Products', OpenBCI Shop. Accessed: Apr. 19, 2026. [Online]. Available: <https://shop.openbci.com/collections/frontpage>
- [31] 'Cerelog ESP-EEG | 8-Channel ADS1299 Biosensing Board for Researchers', Cerealog. Accessed: Apr. 19, 2026. [Online]. Available: https://www.cerealog.com/eeg_researchers.html
- [32] *Cerealog-ESP-EEG/ESP-EEG*. (Apr. 17, 2026). C++. Cerealog ESP-EEG. Accessed: Apr. 19, 2026. [Online]. Available: <https://github.com/Cerealog-ESP-EEG/ESP-EEG>
- [33] 'Cyton Biosensing Board (8-channels)', OpenBCI Shop. Accessed: Apr. 19, 2026. [Online]. Available: <https://shop.openbci.com/products/cyton-biosensing-board-8-channel>
- [34] 'MSP430 Microcontroller Architecture and Applications: A Closer Look at Power Efficiency'. Accessed: Apr. 19, 2026. [Online]. Available: <https://www.ic-components.com/blog/msp430-microcontroller-architecture-and-applications-a-closer-look-at-power-efficiency.jsp>
- [35] T. C. Nguyen, T.-Q. Huynh, T.-T. Vo, P. N. Nguyen, and T. Vo Van, 'An EEG Front-End System Using ADS1299', in *6th International Conference on the Development of Biomedical Engineering in Vietnam (BME6)*, T. Vo Van, T. A. Nguyen Le, and T. Nguyen Duc, Eds, Singapore: Springer, 2018, pp. 717–722. doi: 10.1007/978-981-10-4361-1_123.
- [36] B. Gao, 'Design of portable EEG signal acquisition hardware system based on ADS1299', in *5th International Conference on Information Science, Electrical, and Automation Engineering (ISEAE 2023)*, SPIE, Aug. 2023, pp. 547–553. doi: 10.1117/12.2689432.

- [37] R. Martinek *et al.*, ‘Advanced Bioelectrical Signal Processing Methods: Past, Present and Future Approach—Part I: Cardiac Signals’, *Sensors*, vol. 21, no. 15, p. 5186, Jul. 2021, doi: 10.3390/s21155186.
- [38] T. W. Picton, ‘The P300 wave of the human event-related potential’, *J. Clin. Neurophysiol. Off. Publ. Am. Electroencephalogr. Soc.*, vol. 9, no. 4, pp. 456–479, Oct. 1992, doi: 10.1097/00004691-199210000-00002.
- [39] ‘ADS129x Low-Power, 8-Channel, 24-Bit Analog Front-End for Biopotential Measurements’. Texas Instruments, 2010. Accessed: Apr. 22, 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/ads1298.pdf?ts=1776727133650&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252FADS1298
- [40] ‘AD7779’. Analog Devices. Accessed: Apr. 22, 2026. [Online]. Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/ad7779.pdf>
- [41] ‘AD8422’. Analog Devices. Accessed: Apr. 22, 2026. [Online]. Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/ad8422.pdf>
- [42] ‘STM32 Popular Chip Models Analysis and Selection Guide - Semicon electronics’, 深圳传电电子有限公司. Accessed: May 03, 2026. [Online]. Available: [//www.semicone.com/article-177.html](http://www.semicone.com/article-177.html)
- [43] ‘Best Hardware for Bluetooth Low Energy (BLE) Development 2026’, Argenox. Accessed: May 03, 2026. [Online]. Available: <https://argenox.com/blog/best-hardware-for-bluetooth-low-energy-ble-development-2026>
- [44] S. Laszlo, M. Ruiz-Blondet, N. Khalifian, F. Chu, and Z. Jin, ‘A direct comparison of active and passive amplification electrodes in the same amplifier system’, *J. Neurosci. Methods*, vol. 235, pp. 298–307, Sep. 2014, doi: 10.1016/j.jneumeth.2014.05.012.
- [45] ‘Active EEG Electrode - NVK EEG Electrodes’. Accessed: May 03, 2026. [Online]. Available: <https://www.nvk.com.tw/eeg-lead-wires/active-eeg-electrodes.html>
- [46] ‘Dry Electrode Guide | Wiki’. Accessed: May 03, 2026. [Online]. Available: <https://wiki.mentalab.com/user-guide/dry-electrode-guide/>
- [47] ‘Gold Cup Electrodes’, OpenBCI Shop. Accessed: May 03, 2026. [Online]. Available: <https://shop.openbci.com/products/openbci-gold-cup-electrodes>
- [48] K. E. Mathewson, T. J. L. Harrison, and S. A. D. Kizuk, ‘High and dry? Comparing active dry EEG electrodes to active and passive wet electrodes’, *Psychophysiology*, vol. 54, no. 1, pp. 74–82, Jan. 2017, doi: 10.1111/psyp.12536.
- [49] C. Russomanno, *conorrussomanno/ADS1299*. (Mar. 21, 2026). C++. Accessed: May 04, 2026. [Online]. Available: <https://github.com/conorrussomanno/ADS1299>
- [50] ‘Maximizing BLE Throughput Part 3: Data Length Extension (DLE) – Punch Through’. Accessed: May 04, 2026. [Online]. Available: <https://punchthrough.com/maximizing-ble-throughput-part-3-data-length-extension-dle-2/>
- [51] *OpenBCI/V3_Hardware_Design_Files*. (Apr. 28, 2026). OpenBCI. Accessed: May 05, 2026. [Online]. Available: https://github.com/OpenBCI/V3_Hardware_Design_Files
- [52] electronics notes, ‘Understanding Ceramic Capacitors: MLCC, X7R, CoG, Y5V, NPO...’ Accessed: May 05, 2026. [Online]. Available: https://www.electronics-notes.com/articles/electronic_components/capacitors/ceramic-dielectric-types-cog-x7r-z5u-y5v.php
- [53] ‘X7R, X5R, CoG...: A Concise Guide to Ceramic Capacitor Types - Technical Articles’. Accessed: May 05, 2026. [Online]. Available: <https://www.allaboutcircuits.com/technical-articles/x7r-x5r-cog...-a-concise-guide-to-ceramic-capacitor-types/>
- [54] Yoyo, ‘LDO vs switching regulatorLDO vs Switching Regulator: Which Voltage Regulator to Choose?’, PCBSync. Accessed: May 05, 2026. [Online]. Available: <https://pcbsync.com/lldo-vs-switching-regulator/>
- [55] ‘Why a Strong PCB Ground Plane Is Key to Noise-Free Designs’. Accessed: May 05, 2026. [Online]. Available: <https://jlcpcb.com/blog/pcb-ground-plane-reduces-emi-noise>
- [56] H. W. Ott, *Electromagnetic compatibility engineering*. Hoboken, N.J: John Wiley & Sons, 2009.

- [57] 'Comparing HASL and ENIG Surface Finishes for PCBs'. Accessed: May 05, 2026. [Online]. Available: <https://jlcpcb.com/blog/hasl-vs-enig-surface-finishes>
- [58] P. Konrad, 'The ABC of EMG: A Practical Introduction to Kinesiological Electromyography'. Noraxon INC. [Online]. Available: https://hermanwallace.com/download/The_ABC_of_EMG_by_Peter_Konrad.pdf
- [59] H. J. Hermens, B. Freriks, C. Disselhorst-Klug, and G. Rau, 'Development of recommendations for SEMG sensors and sensor placement procedures', *J. Electromyogr. Kinesiol.*, vol. 10, no. 5, pp. 361–374, Oct. 2000, doi: 10.1016/S1050-6411(00)00027-4.
- [60] Natus Neuro, *10-20 International EEG Electrode Application Video*, (Apr. 04, 2018). Accessed: Apr. 28, 2026. [Online Video]. Available: <https://www.youtube.com/watch?v=b26vNbk8TOc>
- [61] E. A. Ashley and J. Niebauer, 'Conquering the ECG', in *Cardiology Explained*, Remedica, 2004. Accessed: Apr. 28, 2026. [Online]. Available: <https://www.ncbi.nlm.nih.gov/books/NBK2214/>
- [62] P. Kligfield *et al.*, 'Recommendations for the Standardization and Interpretation of the Electrocardiogram', *Circulation*, vol. 115, no. 10, pp. 1306–1324, Mar. 2007, doi: 10.1161/CIRCULATIONAHA.106.180200.
- [63] J. Gomez-Clapers, E. Serrano-Finetti, R. Casanella, and R. Pallas-Areny, 'Can driven-right-leg circuits increase interference in ECG amplifiers?', *Annu. Int. Conf. IEEE Eng. Med. Biol. Soc. IEEE Eng. Med. Biol. Soc. Annu. Int. Conf.*, vol. 2011, pp. 4780–4783, 2011, doi: 10.1109/IEMBS.2011.6091184.
- [64] 'Alpha Wave - an overview | ScienceDirect Topics'. Accessed: May 02, 2026. [Online]. Available: <https://www.sciencedirect.com/topics/neuroscience/alpha-wave>
- [65] O. Jensen and M. Bonnefond, 'The alpha rhythm: from physiology to behavior', *Physiol. Rev.*, vol. 106, no. 3, pp. 1123–1159, Jul. 2026, doi: 10.1152/physrev.00001.2025.
- [66] W. Klimesch, 'Alpha-band oscillations, attention, and controlled access to stored information', *Trends Cogn. Sci.*, vol. 16, no. 12, pp. 606–617, Dec. 2012, doi: 10.1016/j.tics.2012.10.007.
- [67] A. Hartoyo, P. J. Cadusch, D. T. J. Liley, and D. G. Hicks, 'Inferring a simple mechanism for alpha-blocking by fitting a neural population model to EEG spectra', *PLoS Comput. Biol.*, vol. 16, no. 4, p. e1007662, Apr. 2020, doi: 10.1371/journal.pcbi.1007662.
- [68] P. D. Welch, 'The Use of Fast Fourier Transform for the Estimation of Power Spectra: A Method Based on Time Averaging Over Short, Modified Periodograms', *IEEE Trans. Audio Electroacoustics*, Jan. 1967, doi: 10.1109/TAU.1967.1161901.
- [69] J. Samaha and B. R. Postle, 'The Speed of Alpha-Band Oscillations Predicts the Temporal Resolution of Visual Perception', *Curr. Biol.*, vol. 25, no. 22, pp. 2985–2990, Nov. 2015, doi: 10.1016/j.cub.2015.10.007.
- [70] F.-B. Vialatte, M. Maurice, J. Dauwels, and A. Cichocki, 'Steady-state visually evoked potentials: Focus on essential paradigms and future perspectives', *Prog. Neurobiol.*, vol. 90, no. 4, pp. 418–438, Apr. 2010, doi: 10.1016/j.pneurobio.2009.11.005.
- [71] E. Yin, Z. Zhou, J. Jiang, Y. Yu, and D. Hu, 'A Dynamically Optimized SSVEP Brain-Computer Interface (BCI) Speller', *IEEE Trans. Biomed. Eng.*, vol. 62, no. 6, pp. 1447–1456, Jun. 2015, doi: 10.1109/TBME.2014.2320948.
- [72] F. Zenker and J. J. Barajas, 'Auditory P300 development from an active, passive and single-tone paradigms', *Int. J. Psychophysiol.*, vol. 33, no. 2, pp. 99–111, Aug. 1999, doi: 10.1016/S0167-8760(99)00033-1.
- [73] L. A. Farwell and E. Donchin, 'Talking off the top of your head: toward a mental prosthesis utilizing event-related brain potentials', *Electroencephalogr. Clin. Neurophysiol.*, vol. 70, no. 6, pp. 510–523, Dec. 1988, doi: 10.1016/0013-4694(88)90149-6.
- [74] T. Demiralp, A. Ademoglu, M. Schürmann, C. Basar-Eroglu, and E. Basar, 'Detection of P300 Waves in Single Trials by the Wavelet Transform (WT)', *Brain Lang.*, vol. 66, no. 1, pp. 108–128, Jan. 1999, doi: 10.1006/brln.1998.2027.

- [75] 'Waterfall Methodology – Ultimate Guide'. Accessed: May 07, 2026. [Online]. Available: <https://management.org/waterfall-methodology>
- [76] M. Šuštar Habjan *et al.*, 'ISCEV standard for clinical visual evoked potentials (2025 update)', *Doc. Ophthalmol. Adv. Ophthalmol.*, vol. 151, no. 2, pp. 97–112, 2025, doi: 10.1007/s10633-025-10042-1.
- [77] J. P. Rosenfeld, 'P300 in detecting concealed information and deception: A review', *Psychophysiology*, vol. 57, no. 7, p. e13362, Jul. 2020, doi: 10.1111/psyp.13362.
- [78] L. A. Farwell and E. Donchin, 'The truth will out: interrogative polygraphy ("lie detection") with event-related brain potentials', *Psychophysiology*, vol. 28, no. 5, pp. 531–547, Sep. 1991, doi: 10.1111/j.1469-8986.1991.tb01990.x.
- [79] E. R. D. Murphy and J. Rissman, 'Evidence of memory from brain data', *J. Law Biosci.*, vol. 7, no. 1, p. lsaa078, Dec. 2020, doi: 10.1093/jlb/lsaa078.
- [80] 'Tips and tricks for using Lab Streaming Layer (LSL)', Brain Products GmbH. Accessed: May 04, 2026. [Online]. Available: <https://www.brainproducts.com/support-resources/tips-and-tricks-for-lsl/>
- [81] 'Using Recorded OpenBCI Data in OpenViBE', The Autodidacts. Accessed: May 04, 2026. [Online]. Available: <https://www.autodidacts.io/using-recorded-openbci-data-in-openvibe/>

10. APPENDICES

10.1. Appendix A: Project Costing

The complete breakdown of all costs incurred during the project is shown in Table 7. All cost are including VAT. The total cost was £509.17, with £353.19 coming out of the university project budget, £86.48 coming from supervisor budget, and the remaining £69.50 coming from personal budget.

Table 7: Complete bill of materials for the project development

Purchase	Supplier	Budget Source	Quantity	Total Cost
ADS1299 Evaluation Board	Mouser Electronics	University	1	£233.08
PCB Components	Mouser Electronics	University	1	£120.11
Ten20 Conductive Paste	Amazon	Supervisor	1	£71.48
Gold Cup Electrodes	Pinduoduo	Supervisor	8	£15
PCB Manufacture	JLCPCB	Personal	5	£42.34*
LiPo Battery (3.7V 2000mAh)	Amazon	Personal	1	£10.99
ESP32-S3-Devkit-N8R8	The Pi Hut	Personal	1	£10.60
Sseed Studio 106990290 Battery Charger	RS	Personal	1	£5.57
				£509.17

* Including shipping

The university undergraduate labs were used for initial prototyping. In particular the power supplies, digital multi-meters and signal generators were used. The PCB was assembled at GE Aerospace Bristol, making use of their electronics lab and soldering equipment.

A university technician 3D printed the enclosure, on a Formlabs SLA printer. An estimated 30 minutes of time would have been required to put the part on to print, and to subsequently clean and cure the print.

10.2. Appendix B: ADS1299 Communication Library

ADS1299_Definitions.h

```
#ifndef ADS1299_Definitions_h
#define ADS1299_Definitions_h

//SPI Command Definition Byte Assignments (Datasheet, pg. 40)
#define _WAKEUP 0x02    // Wake-up from standby mode
#define _STANDBY 0x04   // Enter Standby mode
#define _RESET 0x06     // Reset the device
#define _START 0x08     // Start and restart (synchronize) conversions
#define _STOP 0x0A      // Stop conversion
#define _RDATAC 0x10    // Enable Read Data Continuous mode (default mode at power-up)
#define _SDATAC 0x11    // Stop Read Data Continuous mode
#define _RDATA 0x12     // Read data by command; supports multiple read back

#define _RREG 0x20       // (also = 00100000) is the first opcode that the address
                        // must be added to for RREG communication
#define _WREG 0x40       // 01000000 in binary (Datasheet, pg. 35)

//Register Addresses (Datasheet, pg. 44)
#define ID 0x00
#define CONFIG1 0x01
#define CONFIG2 0x02
#define CONFIG3 0x03
#define LOFF 0x04
#define CH1SET 0x05
#define CH2SET 0x06
#define CH3SET 0x07
#define CH4SET 0x08
#define CH5SET 0x09
#define CH6SET 0x0A
#define CH7SET 0x0B
#define CH8SET 0x0C
#define BIAS_SENSP 0x0D
#define BIAS_SENSN 0x0E
#define LOFF_SENSP 0x0F
#define LOFF_SENSN 0x10
#define LOFF_FLIP 0x11
#define LOFF_STATP 0x12
#define LOFF_STATN 0x13
#define GPIO 0x14
#define MISC1 0x15
#define MISC2 0x16
#define CONFIG4 0x17

#endif
```

ADS1299.h

```
#ifndef ADS1299_h
#define ADS1299_h

#include <Arduino.h>
#include <SPI.h>
#include "ADS1299_Definitions.h"

class ADS1299 {
public:

    void setup(int _DRDY_PIN, int _CS_PIN, int SCK_PIN, int _MISO_PIN, int
    _MOSI_PIN, int _RESET_PIN, int _START_PIN);

    //ADS1299 SPI Command Definitions (Datasheet, Pg. 35)
```

```

//System Commands
void WAKEUP();
void STANDBY();
void RESET();
void START();
void STOP();

//Data Read Commands
void RDATA();
void SDATA();
void RDATA();

//Register Read/Write Commands
uint8_t RREG(byte _address);
void WREG(byte _address, byte _value); //

void updateData();
void updateDataPacket(uint8_t *dataPacket);

float tCLK, tSDECODE, scaleFactorUV;
int DRDY_PIN, CS_PIN, SCK_PIN, MISO_PIN, MOSI_PIN, RESET_PIN, START_PIN;

int outputCount;

};

#endif

```

ADS1299.cpp

```

#include "ADS1299.h"

SPISettings adsSPI(2000000, MSBFIRST, SPI_MODE1);

void ADS1299::setup(int _DRDY_PIN, int _CS_PIN, int _SCK_PIN, int _MISO_PIN, int
_MOSI_PIN, int _RESET_PIN, int _START_PIN){

    tCLK = 0.489;    // (uS) Assuming default 2.048 MHz clock
    tSDECODE = 5.0; // (uS) tSDECODE > 4 tCLK (Datasheet, pg. 40)
    scaleFactorUV = 45.0 / (24.0 * 8388608.0);
    outputCount = 0;

    // Initialise pins variables
    DRDY_PIN = _DRDY_PIN;
    CS_PIN = _CS_PIN;
    SCK_PIN = _SCK_PIN;
    MISO_PIN = _MISO_PIN;
    MOSI_PIN = _MOSI_PIN;
    RESET_PIN = _RESET_PIN;
    START_PIN = _START_PIN;

    // Initialise pin modes
    pinMode(DRDY_PIN, INPUT);
    pinMode(START_PIN, OUTPUT);
    pinMode(CS_PIN, OUTPUT);
    pinMode(RESET_PIN, OUTPUT);

    digitalWrite(RESET_PIN, HIGH); // Keep ADS1299 out of reset
    digitalWrite(START_PIN, HIGH);
    digitalWrite(CS_PIN, HIGH);    // Deselect the ADS1299 initially

    // Initialise SPI
    SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN, -1);

    delay(1000); // tPOR > 0.128s (Datasheet, pg. 70)
    digitalWrite(RESET_PIN, LOW);
    delayMicroseconds(2); // tRST > 0.977us (Datasheet, pg. 70)

```

```

digitalWrite(RESET_PIN, HIGH);
delay(10); // Wait > 18 tCLK (Datasheet, pg. 70)
}

//System Commands
void ADS1299::WAKEUP() {
    SPI.beginTransaction(adsSPI);
    digitalWrite(CS_PIN, LOW); // Set CS Low to communicate
    delayMicroseconds(tsDECODE);
    SPI.transfer(_WAKEUP);
    delayMicroseconds(tsDECODE); // Must wait 4 or more tCLK cycles before
    taking CS high (Datasheet, pg. 38)
    digitalWrite(CS_PIN, HIGH); // Set CS High to end communication
    SPI.endTransaction();
    delayMicroseconds(tsDECODE);
}

void ADS1299::STANDBY() {
    SPI.beginTransaction(adsSPI);
    digitalWrite(CS_PIN, LOW);
    delayMicroseconds(tsDECODE);
    SPI.transfer(_STANDBY);
    delayMicroseconds(tsDECODE);
    digitalWrite(CS_PIN, HIGH);
    SPI.endTransaction();
    delayMicroseconds(tsDECODE);
}

void ADS1299::RESET() {
    SPI.beginTransaction(adsSPI);
    digitalWrite(CS_PIN, LOW);
    delayMicroseconds(tsDECODE);
    SPI.transfer(_RESET);
    delayMicroseconds(tsDECODE);
    digitalWrite(CS_PIN, HIGH);
    SPI.endTransaction();
    delayMicroseconds(10); // At least 18 tCLK cycles required to execute
    RESET command (Datasheet, pg. 41)
}

void ADS1299::START() {
    SPI.beginTransaction(adsSPI);
    digitalWrite(CS_PIN, LOW);
    delayMicroseconds(tsDECODE);
    SPI.transfer(_START);
    delayMicroseconds(tsDECODE);
    digitalWrite(CS_PIN, HIGH);
    SPI.endTransaction();
    delayMicroseconds(tsDECODE);
}

void ADS1299::STOP() {
    SPI.beginTransaction(adsSPI);
    digitalWrite(CS_PIN, LOW);
    delayMicroseconds(tsDECODE);
    SPI.transfer(_STOP);
    delayMicroseconds(tsDECODE);
    digitalWrite(CS_PIN, HIGH);
    SPI.endTransaction();
    delayMicroseconds(tsDECODE);
}

//Data Read Commands
void ADS1299::RDATA() {
    SPI.beginTransaction(adsSPI);
    digitalWrite(CS_PIN, LOW);
    delayMicroseconds(tsDECODE);

```

```

    SPI.transfer(_RDATAC);
    delayMicroseconds(tsDECODE);
    digitalWrite(CS_PIN, HIGH);
    SPI.endTransaction();
    delayMicroseconds(tsDECODE);
}

void ADS1299::SDATAC() {
    SPI.beginTransaction(adsSPI);
    digitalWrite(CS_PIN, LOW);
    delayMicroseconds(tsDECODE);
    SPI.transfer(_SDATAC);
    delayMicroseconds(tsDECODE);
    digitalWrite(CS_PIN, HIGH);
    SPI.endTransaction();
    delayMicroseconds(tsDECODE);
}

void ADS1299::RDATA() {
    SPI.beginTransaction(adsSPI);
    digitalWrite(CS_PIN, LOW);
    delayMicroseconds(tsDECODE);
    SPI.transfer(_RDATA);
    delayMicroseconds(tsDECODE);
    digitalWrite(CS_PIN, HIGH);
    SPI.endTransaction();
    delayMicroseconds(tsDECODE);
}

uint8_t ADS1299::RREG(byte _address) {
    SPI.beginTransaction(adsSPI);
    digitalWrite(CS_PIN, LOW);
    delayMicroseconds(tsDECODE);

    // Byte 1: 001r rrrr, where r rrrr is the starting register address.
    SPI.transfer(_RREG | _address);
    delayMicroseconds(tsDECODE);

    // Byte 2: 00n nnnn, where n nnnn is the number of registers to read - 1
    SPI.transfer(0x00);
    delayMicroseconds(tsDECODE);

    // Byte 3: Send dummy byte (0x00) to keep SPI clock running.
    uint8_t regValue = SPI.transfer(0x00);
    delayMicroseconds(tsDECODE);

    digitalWrite(CS_PIN, HIGH);
    SPI.endTransaction();
    delayMicroseconds(tsDECODE);

    // Print the result to the Serial Monitor
    Serial.print("Register Value: 0b");
    for (int i = 7; i >= 0; i--) {
        Serial.print(bitRead(regValue, i)); // Read and print each bit from left to
right
    }
    Serial.println("\n");

    return regValue;
}

void ADS1299::WREG(byte _address, byte _value) {
    SPI.beginTransaction(adsSPI);
    digitalWrite(CS_PIN, LOW);
    delayMicroseconds(tsDECODE);

    // Byte 1: 010r rrrr, where r rrrr is the starting register address.
    SPI.transfer(_WREG | _address);

```

```

    delayMicroseconds(tsDECODE);

    // Byte 2: 000n nnnn, where n nnnn is the number of registers to write - 1.
    SPI.transfer(0x00);
    delayMicroseconds(tsDECODE);

    // Byte 3: Register data (in MSB-first format)
    SPI.transfer(_value);
    delayMicroseconds(tsDECODE);

    digitalWrite(CS_PIN, HIGH);
    SPI.endTransaction();
    delayMicroseconds(tsDECODE);

    // Print confirmation to Serial Monitor
    Serial.print("Register 0x");
    Serial.print(_address, HEX);
    Serial.println(" modified. \n");
}

void ADS1299::updateData() {
    // This function assumes that DRDY is LOW and that the ADS1299 is in continuous
    data mode (RDATA)
    SPI.beginTransaction(adsSPI);
    digitalWrite(CS_PIN, LOW);
    delayMicroseconds(tsDECODE);

    long output[9];
    long dataPacket;
    for(int i = 0; i < 9; i++) {
        dataPacket = 0;
        for(int j = 0; j < 3; j++) {
            byte dataByte = SPI.transfer(0x00);
            dataPacket = (dataPacket << 8) | dataByte;
        }

        // Sign extend the 24-bit data to 32 bits
        if (dataPacket & 0x00800000) {
            dataPacket |= 0xFF000000;
        }

        output[i] = dataPacket;
    }

    delayMicroseconds(tsDECODE);
    digitalWrite(CS_PIN, HIGH);
    SPI.endTransaction();
    delayMicroseconds(tsDECODE);
}

void ADS1299::updateDataPacket(uint8_t *dataPacket) {
    // This function assumes that DRDY is LOW and that the ADS1299 is in continuous
    data mode (RDATA)
    SPI.beginTransaction(adsSPI);
    digitalWrite(CS_PIN, LOW);
    delayMicroseconds(tsDECODE);

    // Read all 27 bytes (3 status + 24 channel)
    for(int i = 1; i <= 27; i++) {
        dataPacket[i] = SPI.transfer(0x00);
    }

    delayMicroseconds(tsDECODE);
    digitalWrite(CS_PIN, HIGH);
    SPI.endTransaction();
    delayMicroseconds(tsDECODE);
}

```

10.3. Appendix C: ESP32-S3 Firmware

All firmware was developed in Visual Studio Code, using the PlatformIO extension. Specifically the ESP32 Arduino Core v2.0.17 was used, operating on top of the ESP-IDF v4.4.7 framework.

Platformio.ini

```
[env:esp32-s3-devkitc-1]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
monitor_speed = 115200

board_upload.flash_size = 8MB
board_build.partitions = default_8MB.csv

board_build.arduino.memory_type = qio_opi

build_flags =
    -D ARDUINO_USB_CDC_ON_BOOT=1
    -D ARDUINO_USB_MODE=1
    -D BOARD_HAS_PSRAM
```

Main.cpp

```
#include <Arduino.h>
#include <ADS1299.h>
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>
#include <esp_gap_ble_api.h>

ADS1299 ADS;

// ADS1299 DEFINITIONS
#define RESET_PIN 14
#define DRDY_PIN 9
#define CS_PIN 10
#define MOSI_PIN 11 // AKA Data. Connected to ADS1299 DIN
#define SCK_PIN 12
#define MISO_PIN 13 // AKA Quad. Connected to ADS1299 DOUT
#define START_PIN 18

// BLUETOOTH DEFINITIONS
#define bleServerName "ESP32_EEG_DATA" //BLE server name
#define SERVICE_UUID "91bad492-b950-4226-aa2b-4ede9fa42f59" //BLE service UUID (randomly generated)
#define CHARACTERISTIC_UUID "cba1d466-344c-4be3-ab3f-189f80dd7518" //BLE characteristic UUID (randomly generated)
BLECharacteristic *pCharacteristic = NULL;

// FUNCTION DECLARATIONS
void IRAM_ATTR dataReadyISR(); // DRDY Interrupt
void IRAM_ATTR buttonISR(); // Button Interrupt
void readADSDataTask(void * pvParameters); // Core 1: Read Data from ADS1299
void sendDataTask(void * pvParameters); // Core 0: Send Data to PC

// VARIABLES
SemaphoreHandle_t drdySemaphore; // Semaphore to signal when DRDY goes low
QueueHandle_t packetQueue; // Create a queue to pass 33-byte arrays between cores
uint8_t sampleCounter = 0;
```

```

volatile bool deviceConnected = false;           // Flag to track BLE connection status

// HARDWARE BUTTON DEFINITION
#define BOOT_BUTTON_PIN 0
volatile bool triggerRecordMode = false; // Flag to trigger state change

// -----

//Setup bluetooth callbacks onConnect and onDisconnect
class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer, esp_ble_gatts_cb_param_t *param) {
        deviceConnected = true;
        Serial.println("Device connected!");
        pServer->updateConnParams(param->connect.remote_bda, 6, 12, 0, 400); // Can
change 12 to 10 to force a faster interval.
        esp_ble_gap_set_pkt_data_len(param->connect.remote_bda, 251); // Request data
length extension
    };

    void onDisconnect(BLEServer* pServer) {
        deviceConnected = false;
        Serial.println("Device disconnected. Restarting advertising...");
        // Restart advertising to allow reconnection
        pServer->getAdvertising()->start();
    }
};

// -----

void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.printf("PSRAM size: %u bytes\n", ESP.getPsramSize());

    // Boot button setup
    pinMode(BOOT_BUTTON_PIN, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(BOOT_BUTTON_PIN), buttonISR, FALLING);

    // Initialize Queue holding up to 256 packets of 28 bytes
    packetQueue = xQueueCreate(256, 28 * sizeof(uint8_t));

    // BLUETOOTH LOW ENERGY SETUP
    BLEDevice::init(bleServerName); // Create the BLE
Device
    BLEDevice::setMTU(512); // Set MTU to
maximum to allow for larger packets and reduce fragmentation
    BLEServer *pServer = BLEDevice::createServer(); // Set BLE device
as a server
    pServer->setCallbacks(new MyServerCallbacks()); // Assign callback
function
    BLEService *pService = pServer->createService(SERVICE_UUID); // Start BLE
service with service UUID
    pCharacteristic = pService-
>createCharacteristic(CHARACTERISTIC_UUID, BLECharacteristic::PROPERTY_NOTIFY); //
Create a BLE Characteristic with notify property
    pCharacteristic->addDescriptor(new BLE2902()); // Add a BLE2902
descriptor to allow notifications
    pService->start(); // Start the
service
    BLEAdvertising *pAdvertising = BLEDevice::getAdvertising(); // Start
advertising
    pAdvertising->addServiceUUID(SERVICE_UUID);
    pServer->getAdvertising()->start();
    delay(1000);

    // SETUP ADS1299

```



```

ADS.setup(DRDY_PIN, CS_PIN, SCK_PIN, MISO_PIN, MOSI_PIN, RESET_PIN, START_PIN);
digitalWrite(START_PIN, LOW);    // Pull start low to stop conversations
delay(10);
ADS.SDATAC();                    // Stop continuous data mode to allow register
configuration
ADS.WREG(GPIO, 0b00000000);
ADS.WREG(CONFIG1, 0b10010101);   // Set SPS (110 = 250, 101 = 500, 100 = 1000)
ADS.WREG(CONFIG3, 0b11111110);   // Enable power-down reference buffer, bias
measurement, internal bias ref, bias buffer and bias lead off
ADS.WREG(CONFIG4, 0b00000010);   // Enable lead-off comparator

ADS.WREG(MISC1, 0b00100100);     // Connect SRB1 to inverting inputs.
ADS.WREG(BIAS_SENSP, 0b00000111); // Use positive channel 1 to derive bias
ADS.WREG(BIAS_SENSN, 0b00000001); // Use negative channel 1 to derive bias

ADS.WREG(LOFF, 0b00000100);
ADS.WREG(LOFF_SENSP, 0b11111111); // Enable lead-off detection for channel 1p)
ADS.WREG(LOFF_SENSN, 0b00000001); // Enable lead-off detection for channel 1n)

for(int i=CH1SET; i<=CH8SET; i++) ADS.WREG(i, 0b01100000);

drdySemaphore = xSemaphoreCreateBinary(); //
Create a binary semaphore to signal when DRDY goes low
attachInterrupt(digitalPinToInterrupt(DRDY_PIN), dataReadyISR, FALLING); // Set
up DRDY interrupt on falling edge
ADS.RDATAC();                      // Start continuous data mode
digitalWrite(START_PIN, HIGH);     // Pull start high to begin conversations
delay(10);

// Core 1: Read Data from ADS1299
xTaskCreatePinnedToCore(
    readADSDataTask,
    "ReadTask",
    4096,
    NULL,
    3,
    NULL,
    1);

// Core 0: Send Data to PC
xTaskCreatePinnedToCore(
    sendDataTask,
    "SendTask",
    4096,
    NULL,
    1,
    NULL,
    0);

Serial.println("Setup complete\n");
}

// -----
void loop() {
    // If the BOOT button is pressed, turn off lead off
    if (triggerRecordMode) {
        triggerRecordMode = false; // Reset flag

        // Stop conversations and stop continuous data mode
        digitalWrite(START_PIN, LOW);
        delay(10);
        ADS.SDATAC();

        // 2. Check bias lead-off
        uint8_t config3_val = ADS.RREG(CONFIG3);
        bool biasDisconnected = (config3_val & 0b00000001);

```

```

    if (biasDisconnected) {
        Serial.println("Bias Disconnected");
    } else {
        Serial.println("Bias Connected");
    }

    // Turn off lead off detection
    ADS.WREG(LOFF_SENSP, 0b00000000);
    ADS.WREG(LOFF_SENSN, 0b00000000);
    ADS.WREG(CONFIG3, 0b11111100);

    // Restart continuous data mode
    ADS.RDATAAC();
    digitalWrite(START_PIN, HIGH);
    delay(10);

    Serial.println("Lead-off disabled\n");
    vTaskDelete(NULL);
}

// Delay to yeild to other tasks
delay(100);
}

// -----

void IRAM_ATTR buttonISR() {
    triggerRecordMode = true;
}

// -----

void IRAM_ATTR dataReadyISR() {
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    // Give the semaphore
    xSemaphoreGiveFromISR(drdsSemaphore, &xHigherPriorityTaskWoken);

    // Force a context switch if the read task is ready
    if (xHigherPriorityTaskWoken) {
        portYIELD_FROM_ISR();
    }
}

// -----

void readADSDDataTask(void * pvParameters) {
    uint8_t dataPacket[28];

    for(;;) {
        // Sleep until ISR gives semaphore
        if (xSemaphoreTake(drdsSemaphore, portMAX_DELAY) == pdTRUE) {

            // Assign Sample Counter
            dataPacket[0] = sampleCounter++;

            // Read SPI Data
            ADS.updateDataPacket(dataPacket);

            // Send to the Bluetooth Core
            xQueueSend(packetQueue, dataPacket, 0);
        }
    }
}

```

```

// -----
// -----

void sendDataTask(void * pvParameters) {
    uint8_t singlePacket[28]; // Holds one packet of 28 bytes received from the queue
    uint8_t batchBuffer[224]; // Holds 8 packets of 28 bytes
    int batchCount = 0;

    for(;;) {
        // Clear queue and reset batch if no device is connected
        if (!deviceConnected) {
            xQueueReset(packetQueue);
            batchCount = 0;
            vTaskDelay(pdMS_TO_TICKS(100)); // sleep while disconnected
            continue;
        }

        // Wait for a single 28-byte packet from Core 1
        if (xQueueReceive(packetQueue, &singlePacket, portMAX_DELAY) == pdPASS) {

            // Copy the 28 bytes into the correct slot of 224-byte batch buffer
            memcpy(&batchBuffer[batchCount * 28], singlePacket, 28);
            batchCount++;

            // After 8 packets, send the full 224-byte batch
            if (batchCount == 8) {
                pCharacteristic->setValue(batchBuffer, 224);
                pCharacteristic->notify();

                batchCount = 0;
            }
        }
    }
}

```

10.4. Appendix D: Python Script

The python script has been written in Python 3.12. Core dependencies are:

- Bleak (2.1.1)
- Pyls1 (1.18.1)

Source Code:

```
import asyncio
import math
from bleak import BleakScanner, BleakClient
from pylsl import StreamInfo, StreamOutlet, local_clock

SPS = 500.0
DEVICE_NAME = "ESP32_EEG_DATA"
CHARACTERISTIC_UUID = "cba1d466-344c-4be3-ab3f-189f80dd7518"
scaleFactorUV = 4500000.0 / (24.0 * 8388608.0)

info = StreamInfo('Custom_ESP32_EEG', 'EEG', 8, 500, 'float32',
'esp32_custom_mac_01')
outlet = StreamOutlet(info)

stop_event = asyncio.Event()
packet_queue: asyncio.Queue = asyncio.Queue()

# Shared state for the consumer
class State:
    expected_counter = None
    drop_counter = 0
    last_sample = None # for display

state = State()

def notification_handler(sender, data: bytearray):
    if len(data) == 224:
        packet_queue.put_nowait(bytes(data))

async def consumer_task():
    while not stop_event.is_set():
        try:
            batch = await asyncio.wait_for(packet_queue.get(), timeout=0.5)
        except asyncio.TimeoutError:
            continue

        # Decode all 8 samples into a chunk, then push once
        chunk = []

        for i in range(8):
            packet = batch[i * 28 : (i + 1) * 28]
            sample_counter = packet[0]

            # Drop detection
            if state.expected_counter is not None:
                dropped = (sample_counter - state.expected_counter) % 256
                if dropped > 0:
                    state.drop_counter += dropped
                    # Push current chunk, then insert NaNs
                    if chunk:
                        outlet.push_chunk(chunk)
                        chunk = []
                    nan_sample = [math.nan] * 8
                    outlet.push_chunk([nan_sample] * dropped)
                state.expected_counter = (sample_counter + 1) % 256
```

```

        sample = []
        # Unpack 8-channel sample
        for ch in range(8):
            off = 4 + ch * 3
            raw = (packet[off] << 16) | (packet[off + 1] << 8) | packet[off +
2]
                if raw >= 0x800000:
                    raw -= 0x1000000
                sample.append(raw * scaleFactorUV)
            chunk.append(sample)
            state.last_sample = (sample_counter, sample, packet)

        if chunk:
            outlet.push_chunk(chunk)

    async def display_task():
        while not stop_event.is_set():
            await asyncio.sleep(0.1)
            if state.last_sample is None:
                continue
            sample_counter, voltages, packet = state.last_sample
            c1, c2, c3, c4, c5, c6, c7, c8 = voltages

            status_word = (packet[1] << 16) | (packet[2] << 8) | packet[3]
            p_stat = [(status_word >> (12 + j)) & 1 for j in range(8)]
            ref_stat = (status_word >> 4) & 1

            qsize = packet_queue.qsize()
            dropped_str = f"TOTAL DROPPED : {state.drop_counter} | QUEUE DEPTH :
{qsize}"
            loff_str = f"LEAD-OFF | 1: {p_stat[0]:7} | 2: {p_stat[1]:7} | 3:
{p_stat[2]:7} | 4: {p_stat[3]:7} | 5: {p_stat[4]:7} | 6: {p_stat[5]:7} | 7:
{p_stat[6]:7} | 8: {p_stat[7]:7} | REF: {ref_stat}"
            volts_str = f"VOLTAGES | 1: {c1:7.2f} | 2: {c2:7.2f} | 3: {c3:7.2f} | 4:
{c4:7.2f} | 5: {c5:7.2f} | 6: {c6:7.2f} | 7: {c7:7.2f} | 8: {c8:7.2f}"
            print(f"\033[H\033[J{dropped_str}\n{loff_str}\n{volts_str}", flush=True)

    async def main():
        consumer = asyncio.create_task(consumer_task())
        display = asyncio.create_task(display_task())

        try:
            while not stop_event.is_set():
                print(f"\nScanning for {DEVICE_NAME}...")

                # Scan for device
                esp32_device = await BleakScanner.find_device_by_name(DEVICE_NAME,
timeout=5.0)

                if not esp32_device:
                    print(f"Could not find {DEVICE_NAME}. Retrying in 2 seconds...")
                    await asyncio.sleep(2)
                    continue

                print(f"Found {DEVICE_NAME} at MAC Address: {esp32_device.address}")
                print("Connecting...")

                # Setup the disconnected event
                disconnected_event = asyncio.Event()

                # Pass the callback to BleakClient constructor
                async with BleakClient(esp32_device, disconnected_callback=lambda c:
disconnected_event.set()) as client:
                    print("Connected successfully!")

```

```

print("Subscribing to data stream. Press Ctrl+C to stop.\n\n\n")

# On reconnect, reset state:
state.expected_counter = None
state.drop_counter = 0
state.last_sample = None

# drain stale packets
while not packet_queue.empty():
    packet_queue.get_nowait()

# Subscribe to notifications
await client.start_notify(Characteristic_UUID,
notification_handler)

# Wait until user presses Ctrl+C or the device disconnects
stop_task = asyncio.create_task(stop_event.wait())
disc_task = asyncio.create_task(disconnected_event.wait())
done, pending = await asyncio.wait([stop_task, disc_task],
return_when=asyncio.FIRST_COMPLETED)
for t in pending:
    t.cancel()

# Stop notifying if CTRL+C breakout
if client.is_connected:
    await client.stop_notify(Characteristic_UUID)

state.last_sample = None # Clear display on disconnect

finally:
    consumer.cancel()
    display.cancel()

# Run async loop
if __name__ == "__main__":
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        print("\n\n\nDisconnecting...")
        stop_event.set()

```

10.5. Appendix E: Electrical Schematic

10.6. Appendix F: PCB Bill of Materials

Designator	Manufacturer	Manufacturer Part No.	Quantity	Unit Price	Subtotal Price
C1	Yageo Group	Co603C474K4RACAUTO	1	£0.32	£0.32
C2, C3, C48, C49	Murata	GCJ21BR71C475KA01K	4	£0.28	£1.12
C4, C5	Murata	GCM21BR71C225KA64K	2	£0.17	£0.33
C6, C7, C9, C12, C33	Murata	GRJ21BR61E106KE01L	5	£0.05	£0.51
C10, C11, C35, C36, C38, C39, C40, C43, C44, C46	Yageo Group	CCo603KRX7R7BB105	10	£0.04	£0.37
C13	TDK	CGA3E2X7R1H104K080AD	1	£0.09	£0.09
C14	Murata	GRM1885C2A152JA01D	1	£0.08	£0.08
C15, C16, C17, C18, C19, C20, C21, C22, C23, C24, C25, C26, C27, C28, C29, C30, C31	Yageo Group	Co603C102F5GACTU	17	£0.45	£7.58
C32, C37, C41, C42, C45, C47	Yageo Group	Co603X104J4RACTU	6	£0.10	£1.05
C34	Murata	GRM31CR61A107MEA8L	1	£0.35	£0.35
D1, D2, D3, D4, D5	Texas Instruments	TPD4E1Bo6DCKR	5	£0.72	£3.62
D6	onsemi	SD05T1G	1	£0.33	£0.33
D7	onsemi	ESD5Z5.0T5G	1	£0.17	£0.17
J1	JST	S2B-PH-K-S(LF)(SN)	1	£0.78	£0.78
J2	Molex	217179-0001	1	£0.63	£0.63
J3A, J3B, J3C, J3D, J3E, J3F, J3G, J3H, J3I, J3J, JP1	Samtec	TSW-102-26-F-S	11	£0.23	£2.57
J5	Samtec	TSW-103-07-T-S	1	£0.10	£0.10
R1	Vishay	MCT06030D1303BP100	1	£0.44	£0.44
R2	Yageo Group	RT0603BRD07120KL	1	£0.11	£0.11
R3	Yageo Group	RT0603BRD07105KL	1	£0.19	£0.19
R4	Yageo Group	AT0603BRD07100KL	1	£0.24	£0.24
R5, R6	Yageo Group	RT0603FRE075K1L	2	£0.01	£0.06
R7, R8, R28, R29, R30, R31, R32, R33, R34, R35	Yageo Group	ACo603FR-7W10KL	10	£0.01	£0.08
R9	Vishay	MCT06030C1004FPW00	1	£0.09	£0.09
R10, R11, R12, R13, R14, R15, R16, R17, R18, R19, R20, R21, R22, R23, R24, R25, R26, R27	Vishay	MCT06030D2201BP500	18	£0.06	£1.07
SW1	C&K Components	1101M2S3CQE2	1	£4.19	£4.19
SW2, SW3	Bourns	7914J-1-032	2	£0.80	£1.60
TP1	Keystone	5003	1	£0.26	£0.26
U1	Texas Instruments	LM27762DSSR	1	£1.55	£1.55
U2	Texas Instruments	TLV75733PDBVR	1	£0.25	£0.25
U3	Texas Instruments	TPS7A2033PDBVR	1	£0.24	£0.24
U4	Texas Instruments	ADS1299IPAGR	1	£64.87	£64.87
U5	Espressif	ESP32-S3-WROOM-1-N8R8	1	£4.68	£4.68
					£99.92

10.7. Appendix G: Additional VEP Tests

The following visually evoked potential (VEP) tests have all been conducted following the guidelines set out in the ISCEV Standard for Clinical Visual Evoked Potentials (2025) [76] as much as possible. The example waveforms contained in each section are the examples given by this standard.

10.7.1. Flash VEP Test

Flash VEPs are elicited by a flash subtending at least 20° of the visual field, and are typically recorded from Oz. Once again, the pygame library was used to code the test, creating 50 flashes, occurring at random intervals between 1 and 2 seconds. The resulting data was notch filtered (50 Hz) and bandpassed (0.1-50 Hz), and the responses from every flash were averaged, to produce Figure 27.

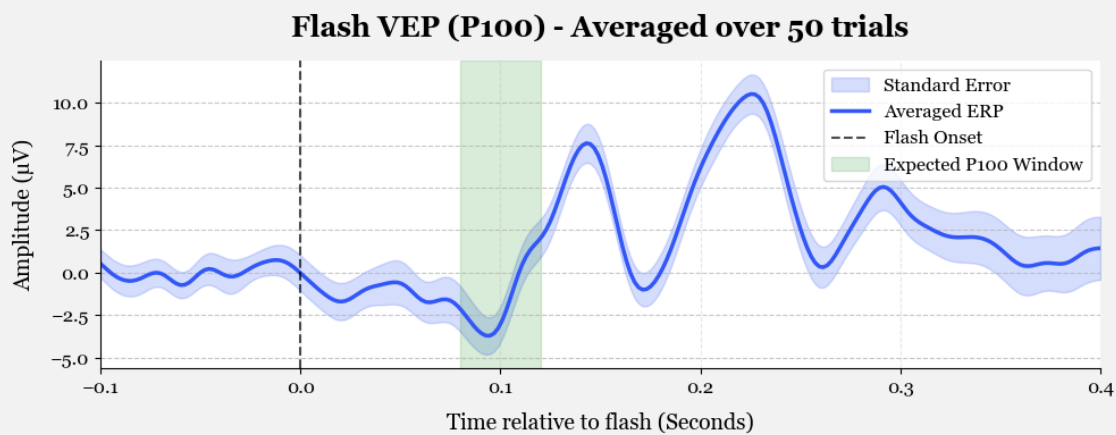


Figure 27: Flash VEP test results

An example flash VEP response is shown in Figure 28, although this data was collected with the reference electrode on Cz, rather than the right earlobe. The recorded data exhibits a similar topology to the example, with a small N1, followed by larger P1, N2 and P2, before finishing with a smaller N3 and P3. The amplitudes of P1 and P2 in the example and recorded data are both approximately 10 μV .

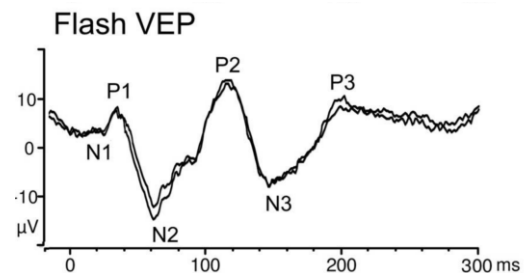


Figure 28: Example flash VEP waveform [76]

However, the amplitude of N2 in the example data is significantly more negative. Moreover, the recorded P1 occurs approximately 100 ms after the example P1, which suggests a temporal offset exists in the recorded data.

10.7.2. Pattern Reversal VEP

The pattern reversal VEP is elicited by an alternating checkerboard pattern, composed of black and white checks, with a constant luminance, and is often used for assessing optic nerve function, as they have a lower inter-subject variability than flash VEPs. This test was recorded, and the data processed identically to the flash VEP test, but with an alternating checkerboard pattern instead of a flash. The averaged results are shown in Figure 29, while an example response is shown in Figure 30.

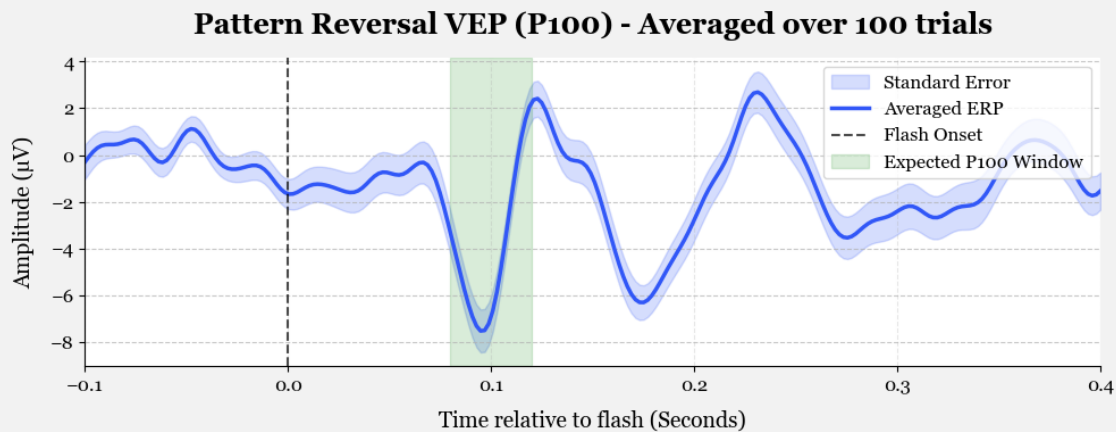


Figure 29: Pattern reversal VEP test results

The recorded data once again exhibits a similar morphology to the example data, with an initial N75, followed by a P100 and an N145.

However, the recorded data has a second positive peak occurring at 240 ms, that does not appear in the example data. Similarly to the flash VEP tests, the recorded peaks are shifted to the right, with the P100 occurring at 125ms rather than 100ms. This reinforces the fact that there is an approximately 25 ms recording offset. A possible solution to this is to use the ESP32-S3 to timestamp the ADS1299 samples as soon as they arrive, rather than letting LSL add timestamps when the data is pushed to the outlet on the host computer.

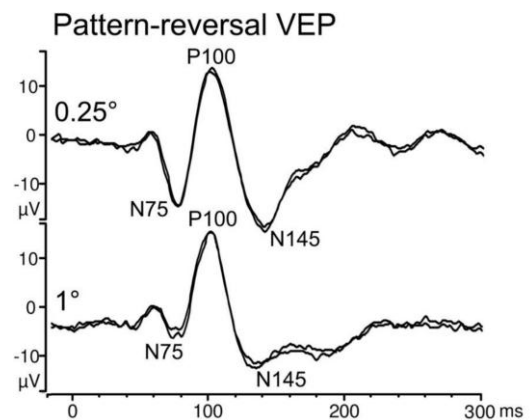


Figure 30: Example pattern reversal VEP waveform [76]

10.7.3. Half Field VEP

The final VEP test conducted was a half field VEP. For this test the alternating checkerboard pattern covered half the screen at a time, whilst the other half of the screen is left blank. 100 reversals were completed on each side of the screen, and as before the recorded data was filtered and averaged. A mistake was made during the stimulus creation for this test, where the left stimulus was marginally larger than the right stimulus. The results of this test are shown in Figure 31.

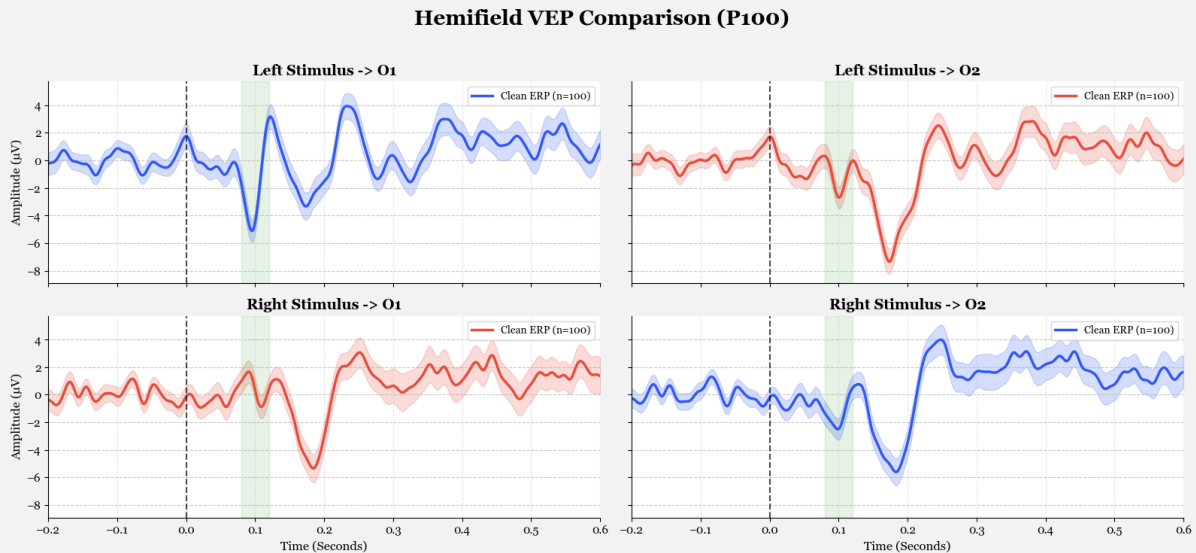


Figure 31: Comparison of O1 and O2 pattern reversal VEP responses from half field stimulation

Figure 31 shows that the N75 and P100 recorded from O1 (left occipital electrode) are much more prominent during the left stimulus than the right stimulus. This is initially surprising, due to the brain's contralateral control, where each hemisphere of the brain is responsible for the opposite side of the body. This would suggest that during left side stimulus, O2 should be the electrode with the stronger response. However, this is not the case, due to paradoxical lateralisation, whereby the response recorded at a lateral scalp location is generated by activity in the contralateral hemisphere of the brain. Therefore, it is expected that O1 would be larger during the left stimulus.

The O2 waveform remained relatively consistent between stimulus sides, this could be attributed to the larger left stimulus.

Generally, the results from this test, particularly the ipsilateral response of O1, help prove the validity of the proposed EEG module.

10.8. Appendix H: Concealed Information Test

Beyond use in a brain computer interface, the P300 wave can also be utilised as a forensic tool for memory detection. As established in Section 5.5, a P300 is elicited when a subject is presented with task related stimuli. It thus follows that when a subject is presented with images bearing no significance to them, alongside images related to their personal experience, the latter will evoke a P300 response. This is known as the concealed information test (CIT) [77].

In this test, a subject was presented with a sequence of 200 coloured squares, each flashed on screen for 200 ms, with a randomised interval of 0.8 to 1.6s. There were a total of six equally occurring colours, each falling into the following three categories:

1. The Probe: A secret colour chosen by the subject, and kept hidden from the experimenter; in this case **Green**.
2. The Target: A designated colour, in this case **Yellow**, which the subject was instructed to count, in order to keep them engaged with the task.
3. The Irrelevants: Four filler colours, bearing no significance to the subject. These were **Red, Blue, Purple and Cyan**.

This setup is known as a three stimulus protocol, and has been carried out to mirror the methods used by Farwell and Donchin (1988) [78]. Figure 32.1 shows the grand averaged and filtered ERP responses for each colour, while Figure 32.2 combines the averages of each of the Irrelevants and plots the standard error of the mean.

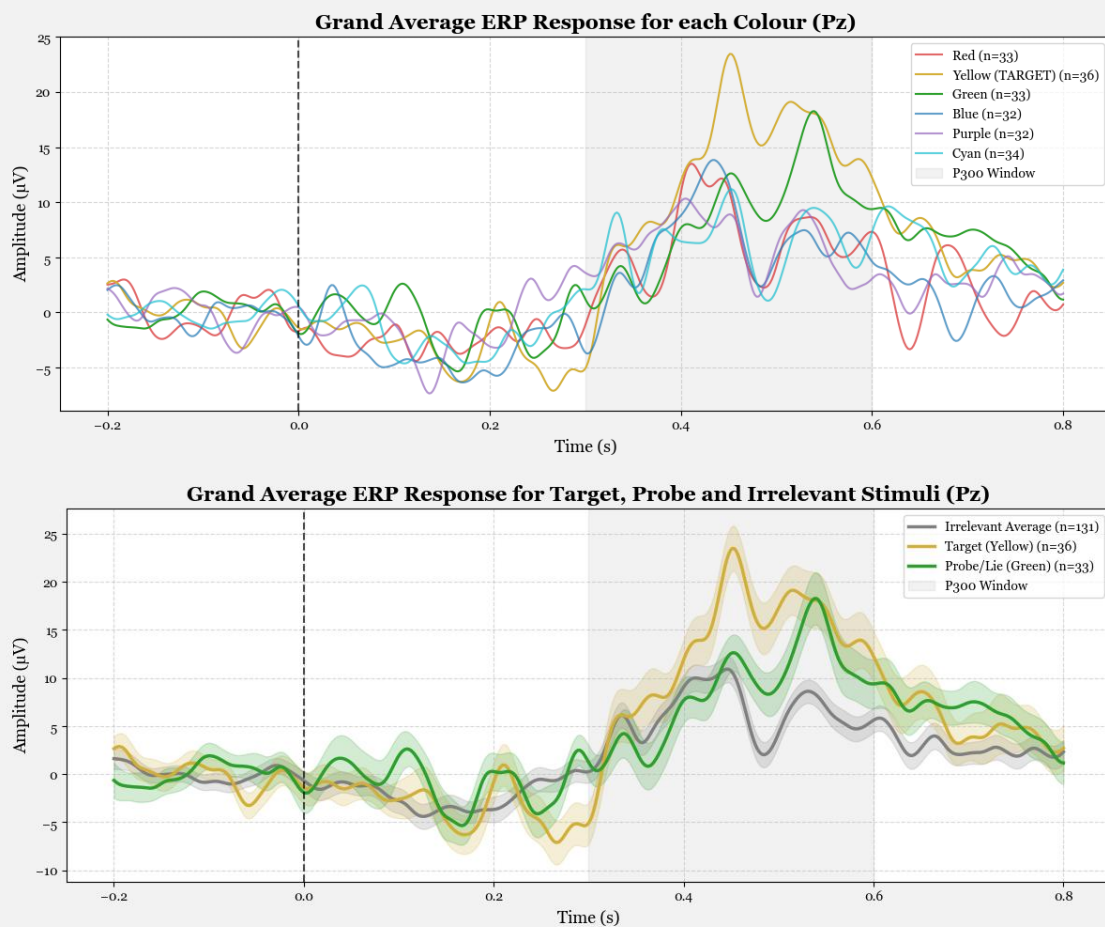


Figure 32: Grand Averaged ERP response from the concealed information test (Probe = Green, Target = Yellow)

As anticipated, the Target colour exhibits the largest P300, as this is the task being actively attended to. Crucially, at 550 ms the Probe response is clearly elevated above the Irrelevant baseline, while each of the Irrelevants have elicited broadly similar responses, thus identifying the Probe as the 'secret colour'. However, it is worth noting that the Probe P300 is delayed by approximately 100ms, and thus does not have particularly similar morphology to the Target P300, which is in contrast to the results obtained by Farwell and Donchin.

Due to the fact that only a single test was conducted, with a single patient, it is difficult to draw concrete conclusions from this test. Additionally, the probe used in the test bears minimal significance to the subject, as it does not relate to a particular memory or experience, being picked immediately before the test began, which could potentially weaken the Probe response.

Finally it is worth noting that it is currently extremely rare for brain based deception detection techniques like this one to be used in court, with applications failing to clear the admissibility threshold due to lack of understood error rates. Additionally these tests cannot distinguish between real and false memories, and are vulnerable to countermeasures being employed by the subject [79].

10.9. Appendix I: LSL Software

Once the data is pushed to an LSL outlet, LSL viewers can be used to visualise the data. For example, BrainVision LSL viewer, shown in Figure 33.1, is a free, real-time oscilloscope style viewer developed by Brain Products. As well as providing basic visualisation functionality, BrainVision can apply high-pass, low-pass and notch filters to the data in real time, as well as displaying channel data in the frequency domain, making it a useful tool for basic testing.

For experiments requiring offline analysis, Lab Recorder can be used to save the EEG data to an XDF (eXtensible Data Format) file, that preserves timestamps, channel metadata and source identifiers for each subscribed stream. Lab Recorder is able to subscribe to multiple streams simultaneously, which is useful when running tests with external stimuli. In these cases, a labelled marker can be pushed to a marker stream when each external stimulus was activated, making it easy to locate the relevant data during analysis. This file can then be opened in python using Pyxdf, or MATLAB using EEGLAB for further analysis, both of which automatically apply a jitter removal algorithm upon import [80].

A final LSL tool that would be particularly useful when using the module in an educational environment is OpenViBE. Rather than writing complex signal-processing code, the user can construct real time data pipelines by dragging and dropping functional blocks, shown in Figure 33.3. These include temporal and spatial filters, as well as tools for Fourier and wavelet analysis, and feature extraction and classification.

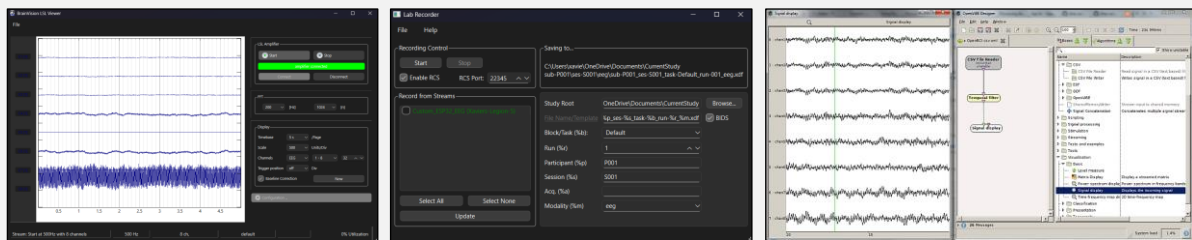


Figure 33: LSL applications: BrainVision (left), Lab Recorder (middle), OpenViBE (right) [81]